AI Transit Pipeline

Complete Installation Guide

COMPONENTS

Pipeline	`ai_transit.sh` · `fetch_repo.sh` · `scan_pipeline.sh`
Reporting	`generate_excel_report.py` · `selfcheck.py`
Air-gap	`prepare_offline_install.sh` · `prepare_offline_cache.sh` · `verify_offline_install.sh`
Container	`docker-run.sh` · `Dockerfile`

Version 3.2 · 2026-11-03

6-layer security gateway for AI-generated repositories

Contents

The shape of it . . . 6
Stage A — Install everything (machine connected) . . . 6
Stage B — Make each tool work offline (machine still connected) . . . 8
Stage C — Disconnect and verify . . . 9
C1 — Scan something real . . . 9
C2 — Keep it working . . . 10
6.1 gitleaks — Secret Detection (Layer 1) . . . 11
6.2 detect-secrets — Entropy Scanning (Layer 1) . . . 12
6.3 ClamAV — Antivirus (Layer 1) . . . 13
6.4 YARA — Custom IOC Rules (Layer 1) . . . 14
6.5 Semgrep — OWASP / CWE / CERT Static Analysis (Layer 2) . . . 15
6.6 trivy — SCA / CVE (Layer 3) . . . 16
6.7 pip-audit — Python CVE (Layer 3) . . . 17
6.8 safety — Python Advisories (Layer 3) . . . 18
6.9 npm + npm audit — Node.js CVE (Layer 3) . . . 18
6.10 Bandit — Python SAST (Layer 5) . . . 19
6.11 ShellCheck — Shell Script SAST (Layer 5) . . . 20
6.12 cppcheck — C/C++ SAST (Layer 5) . . . 21
6.13 hadolint — Dockerfile Linter (Layer 5) . . . 22
6.14 checkov — Terraform / IaC SAST (Layer 5) . . . 23
6.15 ScanCode Toolkit — Licence & Copyright (Layer 6) . . . 24
7.1 Clone from GitHub . . . 25
7.2 Install Python dependencies for report generation . . . 25
7.3 Make scripts executable . . . 25
7.4 Verify scripts . . . 25
7.5 Run the test suite . . . 26
7.6 Generate the integrity manifest . . . 26
9.1 Command-line flags . . . 26
9.2 Environment variables . . . 27
9.3 Per-repo exclusions . . . 27
10.1 Final step — verify the install works offline . . . 29
11.1 Why the scan is offline-only . . . 29
11.2 Per-tool reference . . . 30
11.2.1 Build files — Makefile, .mk, .mak . . . 32
11.2.2 CMake — .cmake and CMakeLists.txt . . . 32
11.2.3 HTML — .html, .htm, .xhtml . . . 33

11.2.4 PDF scanning — no staging, and why that matters . . . 34
11.2.5 Traversal and key material — the rules that run on every file . . . 35
11.2.6 XML, Maven and Ant . . . 36
11.2.7 Package manifests — package.json, composer.json, requirements.txt . . . 37
11.2.8 Gradle — *.gradle, *.gradle.kts, gradle.properties, gradle-wrapper.properties . . . 37
11.2.9 setup.py and conanfile.py . . . 38
11.2.10 Known credential formats — the rule that runs on every file . . . 39
11.2.11 PowerShell — every extension . . . 40
11.2.12 Windows batch — .bat and .cmd . . . 41
11.2.13 Java archives — .jar, .war, .ear, .aar . . . 42
11.2.14 Office documents — OOXML, OpenDocument, legacy OLE2 and RTF . . . 44
11.3 Environment variables . . . 46
11.4 Building the cache — on a connected host . . . 46
11.5 Transferring and verifying . . . 46
11.6 Running . . . 47
11.7 Confirming a clean offline run . . . 47
11.8 Coverage — proving the scan actually ran . . . 47
11.9 Keeping the cache current . . . 48
12.1 Choose a path . . . 49
12.2 Path A — Docker image transfer . . . 49
12.3 Path B — Native package staging . . . 50
12.4 Verify the installation . . . 51
12.5 Then stage the scan data . . . 51
13.1 Quick smoke test (local directory) . . . 51
13.2 FAIL test — embedded secret . . . 52
13.3 FAIL test — SQL injection (OWASP A03) . . . 53
13.4 FAIL test — vulnerable dependency (CVE) . . . 53
13.5 WARN test — risky licence (GPL) . . . 53
13.6 FAIL test — Dockerfile misconfigurations . . . 53
13.7 Scan a real public GitHub repository . . . 54
13.8 Run the self-check to verify the pipeline itself . . . 54
13.9 CI mode — quiet verdict with severity filter . . . 54
13.10 Diff mode — scan only changed files (PR workflow) . . . 54
13.11 Private repository scan . . . 55
13.12 Report-only mode (audit without blocking) . . . 55
13.13 Air-gapped scan . . . 55
14.1 Output formats and selective checks . . . 55
14.2 The bundle manifest — generate it at install time . . . 55

Dedicated service account 56
Network isolation (allow only github.com outbound) 56
Quarantine filesystem (noexec) 56
Weekly tool updates (cron) 56
Log rotation 56
17.1 What each layer covers 58
17.2 Fixtures 59
17.3 Adding a rule 59
19.1 Build arguments 60
19.2 Why the digests matter 60
19.3 Running through the wrapper 61

1. Overview

LAYER	DETECTS	TOOLS	FAILS ON
L1	**Secrets & malware**	gitleaks · detect-secrets · ClamAV · YARA	**credential or malware found**
L2	**OWASP Top 10 · CWE Top 25**	Semgrep (4 rulesets, one pass)	**ERROR / WARNING finding**
L3	**Dependency CVEs**	trivy · pip-audit · safety · npm audit	**HIGH / CRITICAL CVE**
L4	**Built-in patterns**	grep rules — always available, no tool needed	**CWE-798 / 22 / 918 / 327 / 338**
L5	**Per-language SAST**	Bandit · ShellCheck · cppcheck · hadolint · checkov	**tool-specific finding**
L6	**Licence & package CVE**	ScanCode Toolkit	**risky licence → WARN · CVE → FAIL**

Any layer may FAIL. One FAIL quarantines the repository — the verdict is PASS only when none do.

Figure 1 — the six scanning layers, the tools in each, and what makes each one fail.

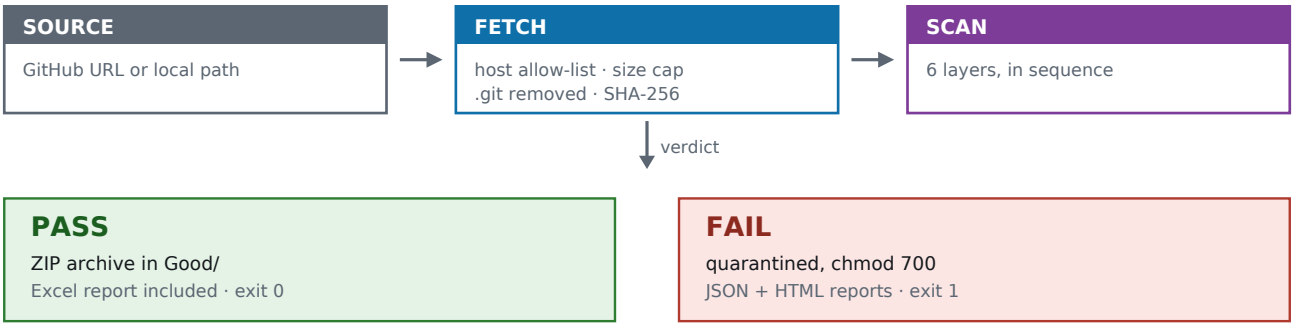


Figure 2 — end-to-end flow from source to verdict.

The AI Transit Pipeline is a 6-layer security gateway that scans AI-generated code before enterprise import.

Layer	Tools	Blocks on
L1 — Secrets & AV	gitleaks, detect-secrets, ClamAV, YARA	Secret/credential/malware detected
L2 — OWASP/CWE	Semgrep	ERROR/WARNING severity finding
L3 — SCA/CVE	trivy, pip-audit, safety, npm audit	HIGH/CRITICAL CVE in dependency
L4 — Patterns	grep (built-in)	CWE-798/22/918/327/338 match

Layer	Tools	Blocks on
L5 — Per-type SAST	Bandit, ShellCheck, cppcheck, hadolint, checkov, Semgrep per-lang	Tool-specific finding (also covers Rust, Kotlin, C#)
L6 — Licence	ScanCode Toolkit	CRITICAL/HIGH CVE in detected package

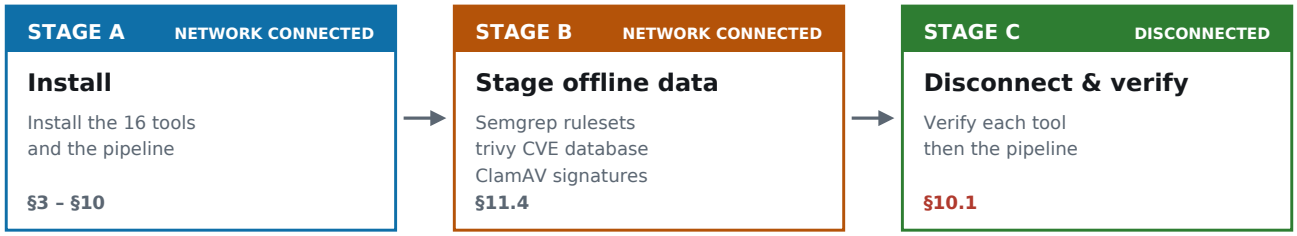
Verdict: any FAIL in any layer → package quarantined. WARNs are logged but do not block.

2. End-to-End Walkthrough — Blank Machine to Verified Offline Install

This is the spine of the manual. It takes one machine from bare OS to a fully offline-capable installation that has been *proved* to work disconnected.

Later sections are the detail behind each step; follow this one and refer to them as directed.

The shape of it



Stage C is not optional.
Until it has run, “offline-ready” is an assumption — and the failure it guards against is silent.

Figure 3 — the three stages of the walkthrough, and the network state each runs in.

```
┌─ Stage A ──────────┐   ┌─ Stage B ──────────┐   ┌─ Stage C ──────────┐
│  CONNECTED         │   │  CONNECTED         │   │  DISCONNECTED      │
│                    │   │                    │   │                    │
│  Install the 16    │──▶│  Stage the data    │──▶│  Pull the cable    │
│  tools + pipeline  │   │  each tool reads   │   │  Verify per tool   │
│                    │   │  offline           │   │  Verify pipeline   │
└────────────────────┘   └────────────────────┘   └────────────────────┘
```

The machine is connected for A and B, then disconnected for C. **Stage C is not optional.** Until it has run, "offline-ready" is an assumption, and the failure mode it protects against — a scan that reports PASS having examined nothing — is silent.

Building on one machine and deploying to another? Do Stages A and B here, then §12 covers packaging and transferring the result. Stage C is run on the target.

Stage A — Install everything (machine connected)

Step	Action	Section
A1	Confirm the OS, hardware and bash version	§3
A2	Install the system packages	§4
A3	Create the Python virtual environment	§5
A4	Install the 16 tools, one at a time	§6
A5	Install the pipeline scripts	§7
A6	Create the working directories	§8
A7	Run the full verification	§10

Condensed, for a fresh Ubuntu 22.04 / 24.04 host:

```
# A2 — system packages
sudo apt-get update && sudo apt-get install -y \
    bash git curl ca-certificates wget gpg rsync \
    jq zip unzip file coreutils \
    python3 python3-pip python3-venv build-essential \
    nodejs npm shellcheck cppcheck clamav clamav-daemon clamav-freshclam yara

# A3 — virtual environment
python3 -m venv /opt/ai-transit/venv
source /opt/ai-transit/venv/bin/activate

# A4 — Python tools
pip install --upgrade pip
pip install openpyxl reportlab python-docx detect-secrets bandit \
            pip-audit safety semgrep checkov scancode-toolkit

# A4 — Go tool. gitleaks needs Go 1.24.11 or newer: install Go from https://go.dev/dl/
# (the distribution's golang-go package is far too old on Ubuntu 22.04 and Debian 12)
CGO_ENABLED=0 go install github.com/zricethezav/gitleaks/v8@latest
sudo cp ~/go/bin/gitleaks /usr/local/bin/

# A4 — standalone binaries (pin versions; see §19.1 for digests)
curl -sfL "https://github.com/aquasecurity/trivy/releases/download/v0.58.2/trivy_0.58.2_Linux-64bit.tar.gz" \
    | sudo tar -xzf - -C /usr/local/bin trivy
sudo curl -sSL "https://github.com/hadolint/hadolint/releases/download/v2.12.0/hadolint-Linux-x86_64" \
    -o /usr/local/bin/hadolint && sudo chmod +x /usr/local/bin/hadolint

# A4 — ClamAV signatures (needed now; staged again in Stage B)
sudo systemctl stop clamav-freshclam 2>/dev/null || true
sudo freshclam
sudo systemctl start clamav-freshclam 2>/dev/null || true

# A5/A6 — pipeline and directories
sudo mkdir -p /opt/ai-transit/{fetch,quarantine,approved,reports,logs,yara-rules}
sudo chmod 700 /opt/ai-transit/quarantine
```

A7 — confirm every tool is actually present before going further:

```
for t in gitleaks detect-secrets clamscan yara semgrep trivy \
         bandit shellcheck cppcheck hadolint checkov scancode; do
    command -v "$t" >/dev/null && echo "  present : $t" || echo "  MISSING : $t"
done
./tests/run_tests.sh          # Expected: ✔ 740/740 passed
```

A tool missing here will still be missing offline, where it is much harder to notice. Resolve every `MISSING` before continuing.

Stage B — Make each tool work offline (machine still connected)

Eight of the tools already work offline; five need data staged first; four cannot work offline at all. This is the table that matters:

Tool	Layer	Offline requirement	How it is satisfied
gitleaks	L1	None — rules compiled in	—
detect-secrets	L1	None — rules embedded	—
ClamAV	L1	**Signature database**	`freshclam`, then signatures copied to the cache
YARA	L1	Your own `.yar` files	Copy them to `$WORK_DIR/yara-rules/`
Semgrep	L2	**Ruleset YAML per ruleset**	`semgrep --config p/<name> --dump-config`
trivy	L3	**Vulnerability database**	`trivy fs --download-db-only --cache-dir`
pip-audit	L3	*Impossible* — remote service	Covered by trivy instead
safety	L3	*Impossible* — remote service	Covered by trivy instead
npm audit	L3	*Impossible* — remote registry	Covered by trivy instead
grep rules	L4	None — built in	—
Bandit	L5	None — local AST analysis	—
ShellCheck	L5	None	—
cppcheck	L5	None	—
hadolint	L5	None	—
checkov	L5	None, but must not fetch schemas	`--skip-download`, applied automatically
ScanCode	L6	None for licence/copyright	`--vulnerability` dropped automatically

One command stages everything in the middle column:

```
./prepare_offline_cache.sh /opt/ai-transit/offline-cache
```

Read its summary. It reports what it staged and what it could not, and a skipped item here becomes a silent gap later.

Verify the staged data is plausible before disconnecting:

```
cd /opt/ai-transit/offline-cache
ls -1 semgrep-rules/     # expect 5 .yaml files
du -sh trivy-db/         # expect several hundred MB — a few KB means it failed
ls -1 clamav/            # expect *.cvd or *.cld
date -u +%Y-%m-%d > .cache_built_on
```

Set the working directories once. The scan itself is always offline (there is no online mode); `--offline` only makes `ai_transit.sh` refuse a remote URL:

```
cat >> ~/.bashrc <<'EOF'
export WORK_DIR=/opt/ai-transit
export OFFLINE_CACHE=/opt/ai-transit/offline-cache
```

```
EOF
source ~/.bashrc
```

Stage C — Disconnect and verify

Disconnect the machine. Physically unplug it, or:

```
sudo ip link set "$(ip route | awk '/default/{print $5; exit}')" down
```

Then run the verification, which checks each tool individually before running the pipeline:

```
./verify_offline_install.sh
```

Per-tool checking matters because the pipeline degrades quietly: a tool that cannot work offline produces an empty result, and an empty result is indistinguishable from a clean one. Checking tools one at a time turns that silence into a named failure.

Expected output on a correctly prepared machine:

```
── Network state
  ✔ Network is unreachable — this is a genuine offline test.

── Group A — no staged data required
  ✔ gitleaks works offline
  ✔ detect-secrets works offline
  ✔ bandit works offline
  ...

── Group B — requires staged data
  ✔ semgrep runs from staged rules (5 ruleset file(s))
  ✔ semgrep cannot reach its registry (expected offline)
  ✔ trivy scans from the staged database
  ✔ clamscan runs against signatures in /opt/ai-transit/offline-cache/clamav
  ...

── End-to-end — pipeline scan with coverage check
  ✔ offline pipeline run completed
      [OK ] L2_owasp_cwe               ran
      [OK ] L3_dependency_cve          ran
      ...
  ✔ all required layers ran offline

  ✔  13/13 checks passed
```

Exit status `0` means the installation is offline-ready. Any failure names the tool and points at the staging step that fixes it.

Cannot disconnect the machine? `--simulate` forces outbound HTTP to a dead port, which catches the common case:

```
./verify_offline_install.sh --simulate
```

It is weaker than a real disconnection — it does not defeat cached DNS or a local proxy — so treat a pass as provisional until you have tested genuinely disconnected.

C1 — Scan something real

```
./ai_transit.sh --offline /path/to/some/repo
```

```
REPORT=$(ls -t "$WORK_DIR"/reports/report_*.json | head -1)
python3 -c "
import json,sys
d=json.load(open(sys.argv[1]))
print('verdict :', d['verdict'])
print('gaps    :', ', '.join(d['coverage_gaps']) or 'none')
" "$REPORT"
```

`L1_ioc_yara` is a legitimate gap if you have written no custom IOC rules. Everything else should be absent. §11.8 explains the coverage block; the step-by-step operating procedure is **OFFLINE_RUNBOOK.md**.

C2 — Keep it working

The install is static, but the staged data is not. A stale CVE database reports clean for everything published since it was built — the same outcome as not scanning, and harder to notice.

Reconnect, re-run `prepare_offline_cache.sh`, disconnect, re-run `verify_offline_install.sh`. **Weekly** for the trivy database and ClamAV signatures; monthly is sufficient for Semgrep rulesets. See §11.9.

3. Prerequisites

Supported platforms: Ubuntu 22.04 LTS / Debian 12 (production), Ubuntu 24.04 LTS (recommended). Windows users: install via **WSL2** (see Section 4 of base guide).

Minimum hardware: 4-core CPU, 8 GB RAM, 20 GB free disk.

4. Core System Packages

Install the baseline packages first. These are required for the pipeline to run at all.

```
sudo apt-get update && sudo apt-get upgrade -y

sudo apt-get install -y \
    bash git curl ca-certificates wget gpg \
    jq zip unzip file coreutils \
    python3 python3-pip python3-venv \
    build-essential \
    nodejs npm
```

Verify:

```
bash   --version | head -1   # expect: GNU bash, version 5.x
git    --version             # expect: git version 2.x
python3 --version            # expect: Python 3.10+
go     version               # expect: go1.21+
node   --version             # expect: v18+ or v20+
jq     --version             # expect: jq-1.6+
```

5. Python Virtual Environment

Using a dedicated venv avoids conflicts with system packages and makes the installation self-contained.

```
python3 -m venv /opt/ai-transit/venv
echo 'source /opt/ai-transit/venv/bin/activate' >> ~/.bashrc
source /opt/ai-transit/venv/bin/activate
pip install --upgrade pip
```

> All `pip install` commands in this guide assume the venv is active.

Verify:

```
which python3     # must show /opt/ai-transit/venv/bin/python3
python3 --version # Python 3.10+
```

6. Tool-by-Tool Installation

Each section covers: finding the latest version, installing, verifying, offline setup, and a functional test.

6.1 gitleaks — Secret Detection (Layer 1)

What it does: scans all file types for secrets, API keys, credentials and tokens. The pipeline runs it on the directory, not on git history (`--no-git`).

Find the latest version

```
# Query the Go module proxy for the latest release
curl -s https://proxy.golang.org/github.com/zricethezav/gitleaks/v8/@latest | jq -r '.Version'
```

Install (Go — recommended, produces a static binary with `CGO_ENABLED=0`)

```
CGO_ENABLED=0 go install github.com/zricethezav/gitleaks/v8@latest
sudo cp ~/go/bin/gitleaks /usr/local/bin/gitleaks
```

Use the module path exactly as written: it is the one declared in gitleaks' `go.mod`, and `go install` refuses `github.com/gitleaks/gitleaks/v8`. v8.30.1 needs Go 1.24.11 or newer; a Go 1.21+ toolchain downloads the newer one on its own.

Install (Homebrew — macOS / Linux with brew)

```
brew install gitleaks
```

Verify installation

```
gitleaks version
# A release build prints its version. A build made with `go install` prints:
#   version is set by build process
```

Offline operation

gitleaks has **no external database** — all detection rules are embedded in the binary. Once installed, it works fully offline.

Functional test

```
# Create a test file with a fake AWS access key id. It is assembled at run time so
# that this document never holds a contiguous token for a secret scanner to flag.
mkdir /tmp/gl-test
printf 'aws_access_key_id = "AKIA%s"\n' "QYLPMN5HHHFPZAM2" > /tmp/gl-test/config.py

gitleaks detect --no-git --source /tmp/gl-test -v
# Expected: exit code 1, finding reported for config.py

# Clean file — should PASS
echo 'import os; key = os.environ["AWS_ACCESS_KEY_ID"]' \
    > /tmp/gl-test/config_clean.py
gitleaks detect --no-git --source /tmp/gl-test/config_clean.py -v
# Expected: exit code 0

rm -rf /tmp/gl-test
```

AWS's own documentation key (`AKIAIOSFODNN7EXAMPLE`) is deliberately allowlisted by gitleaks and by the pipeline, so it is not a usable test value.

6.2 detect-secrets — Entropy Scanning (Layer 1)

What it does: complements gitleaks with Shannon entropy analysis and regex-based patterns to find high-entropy strings that look like secrets.

Find the latest version

```
pip index versions detect-secrets 2>/dev/null | head -1
# or:
curl -s https://pypi.org/pypi/detect-secrets/json | jq -r '.info.version'
```

Install

```
pip install detect-secrets
```

Verify installation

```
detect-secrets --version
# Expected: x.y.z
```

Offline operation

No external database. Works fully offline once installed.

Functional test

```
mkdir /tmp/ds-test
echo 'password = "Sup3rS3cr3tP@ssw0rd!"' > /tmp/ds-test/app.py

detect-secrets scan /tmp/ds-test/app.py
# Expected: JSON with non-empty "results" containing app.py

echo 'password = os.getenv("PASSWORD")' > /tmp/ds-test/app_clean.py
detect-secrets scan /tmp/ds-test/app_clean.py
# Expected: JSON with empty "results" {}
```

```
rm -rf /tmp/ds-test
```

6.3 ClamAV — Antivirus (Layer 1)

What it does: scans files against a database of known malware signatures. Requires regular DB updates to stay effective.

Find the latest version

```
apt-cache policy clamav | grep Candidate
# or for the latest upstream:
curl -s https://www.clamav.net/downloads | grep -oP 'clamav-\K[0-9.]+(?=\.tar)' | head -1
```

Install

```
sudo apt-get install -y clamav clamav-daemon clamav-freshclam
```

Update virus database (required before first use)

```
sudo systemctl stop clamav-freshclam
sudo freshclam
sudo systemctl start clamav-freshclam
sudo systemctl enable clamav-freshclam
```

Verify installation

```
clamscan --version
# Expected: ClamAV x.y.z/NNNNN/...
freshclam --version
# Expected: ClamAV x.y.z
```

Offline operation

ClamAV works offline once the virus DB is downloaded. To prepare for offline use:

```
# On a machine with internet access, download the DB files
sudo freshclam

# The DB files are stored at (copy these to the offline machine):
ls /var/lib/clamav/
# main.cvd (or main.cld), daily.cvd (or daily.cld), bytecode.cvd

# On the offline machine, copy the DB files to /var/lib/clamav/
# then reload:
sudo systemctl restart clamav-daemon
```

Functional test

```
# ClamAV includes the EICAR test signature — a safe test virus string
echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' \
    > /tmp/eicar.txt

clamscan /tmp/eicar.txt
# Expected: /tmp/eicar.txt: Eicar-Signature FOUND
# Expected exit code: 1

clamscan /bin/ls
```

```
# Expected: /bin/ls: OK
# Expected exit code: 0

rm /tmp/eicar.txt
```

6.4 YARA — Custom IOC Rules (Layer 1)

What it does: loads organisation-specific YARA rules from `yara-rules/` and matches them against every file. Fully customisable pattern engine.

Find the latest version

```
apt-cache policy yara | grep Candidate
# or from GitHub:
curl -s https://api.github.com/repos/VirusTotal/yara/releases/latest \
    | jq -r '.tag_name'
```

Install

```
sudo apt-get install -y yara
```

Verify installation

```
yara --version
# Expected: x.y.z
```

Offline operation

YARA has no external database — all rules are local `.yar` files. Works fully offline.

Create a minimal test rule

```
mkdir -p /opt/ai-transit/yara-rules

cat > /opt/ai-transit/yara-rules/test.yar << 'EOF'
rule Suspicious_Base64_Exec {
    meta:
        description = "Detects base64-encoded exec calls"
    strings:
        $b64_exec = /eval\(base64_decode\(/ nocase
    condition:
        $b64_exec
}
EOF
```

Functional test

```
echo '<?php eval(base64_decode("dW5saW5r..."); ?>' > /tmp/test.php

yara /opt/ai-transit/yara-rules/test.yar /tmp/test.php
# Expected: Suspicious_Base64_Exec /tmp/test.php

echo '<?php echo "Hello World"; ?>' > /tmp/clean.php
yara /opt/ai-transit/yara-rules/test.yar /tmp/clean.php
# Expected: (no output — no match)

rm /tmp/test.php /tmp/clean.php
```

6.5 Semgrep — OWASP / CWE / CERT Static Analysis (Layer 2)

What it does: runs four security rulesets (OWASP Top 10, CWE Top 25, security-audit, secrets) across all supported languages using pattern matching on the AST.

Find the latest version

```
curl -s https://pypi.org/pypi/semgrep/json | jq -r '.info.version'
```

Install

```
pip install semgrep
```

Verify installation

```
semgrep --version
# Expected: 1.x.x
```

Offline operation

Semgrep needs internet access **only on the first run** to download rules. Prepare for offline:

```
# On a machine with internet, pre-download rules to a local directory
mkdir -p /opt/ai-transit/semgrep-rules
semgrep --config p/owasp-top-ten   --dump-config > /opt/ai-transit/semgrep-rules/owasp.yaml
semgrep --config p/cwe-top-25      --dump-config > /opt/ai-transit/semgrep-rules/cwe.yaml
semgrep --config p/security-audit  --dump-config > /opt/ai-transit/semgrep-rules/audit.yaml
semgrep --config p/secrets         --dump-config > /opt/ai-transit/semgrep-rules/secrets.yaml

# On the offline machine, run against local rules:
semgrep --config /opt/ai-transit/semgrep-rules/ <repo_dir>
```

Functional test

```
cat > /tmp/test_sqli.py << 'EOF'
import sqlite3
def get_user(username):
    conn = sqlite3.connect("db.sqlite3")
    query = "SELECT * FROM users WHERE name = '" + username + "'"
    return conn.execute(query).fetchall()
EOF

semgrep --config p/owasp-top-ten --json /tmp/test_sqli.py \
    | jq '.results | length'
# Expected: 1 or more (SQL injection detected)

cat > /tmp/test_clean.py << 'EOF'
import sqlite3
def get_user(username):
    conn = sqlite3.connect("db.sqlite3")
    return conn.execute("SELECT * FROM users WHERE name = ?", (username,)).fetchall()
EOF

semgrep --config p/owasp-top-ten --json /tmp/test_clean.py \
    | jq '.results | length'
# Expected: 0

rm /tmp/test_sqli.py /tmp/test_clean.py
```

6.6 trivy — SCA / CVE (Layer 3)

What it does: scans dependency manifests (requirements.txt, package-lock.json, go.sum, pom.xml, Cargo.lock …) against NVD, OSV and GitHub Advisory databases.

Find the latest version

```
curl -s https://api.github.com/repos/aquasecurity/trivy/releases/latest \
    | jq -r '.tag_name'
```

Install (official script — recommended)

```
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
    | sudo sh -s -- -b /usr/local/bin
```

Install (apt repository)

```
sudo apt-get install -y wget gnupg
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key \
    | sudo gpg --dearmor -o /usr/share/keyrings/trivy.gpg
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] \
    https://aquasecurity.github.io/trivy-repo/deb generic main" \
    | sudo tee /etc/apt/sources.list.d/trivy.list
sudo apt-get update && sudo apt-get install -y trivy
```

Verify installation

```
trivy --version
# Expected: Version: x.y.z
```

Download vulnerability database (required before first offline use)

```
# Pre-download the CVE database (requires internet)
trivy image --download-db-only
trivy image --download-java-db-only

# DB is cached at:
ls ~/.cache/trivy/db/
```

Offline operation

```
# After downloading the DB, run offline:
trivy fs <repo_dir> \
    --scanners vuln \
    --severity HIGH,CRITICAL \
    --skip-db-update \
    --offline-scan
```

Functional test

```
mkdir /tmp/trivy-test

# Create a requirements.txt with a known vulnerable package
cat > /tmp/trivy-test/requirements.txt << 'EOF'
Pillow==9.0.0
requests==2.18.0
EOF

trivy fs /tmp/trivy-test \
```

```
    --scanners vuln \
    --severity HIGH,CRITICAL \
    --format table
# Expected: findings for known CVEs in Pillow 9.0.0 and requests 2.18.0

# Clean requirements
cat > /tmp/trivy-test/requirements_clean.txt << 'EOF'
Pillow==10.3.0
requests==2.32.3
EOF

trivy fs /tmp/trivy-test/requirements_clean.txt \
    --scanners vuln \
    --severity HIGH,CRITICAL \
    --format table
# Expected: No vulnerabilities found

rm -rf /tmp/trivy-test
```

6.7 pip-audit — Python CVE (Layer 3)

What it does: audits Python requirements files against the Python Packaging Advisory Database (PyPA) and OSV.

Find the latest version

```
curl -s https://pypi.org/pypi/pip-audit/json | jq -r '.info.version'
```

Install

```
pip install pip-audit
```

Verify installation

```
pip-audit --version
# Expected: pip-audit x.y.z
```

Offline operation

pip-audit requires internet to query the PyPA/OSV database. For offline use, use trivy with `--offline-scan` instead (covers the same packages).

Functional test

```
cat > /tmp/req_vuln.txt << 'EOF'
Pillow==9.0.0
requests==2.18.0
EOF

pip-audit -r /tmp/req_vuln.txt
# Expected: one or more vulnerabilities listed, exit code 1

cat > /tmp/req_clean.txt << 'EOF'
Pillow==10.3.0
requests==2.32.3
EOF

pip-audit -r /tmp/req_clean.txt
```

```
# Expected: No known vulnerabilities found, exit code 0

rm /tmp/req_vuln.txt /tmp/req_clean.txt
```

6.8 safety — Python Advisories (Layer 3)

What it does: checks Python dependencies against the Safety DB (PyUp.io), a curated advisory database with additional entries not always in OSV.

Find the latest version

```
curl -s https://pypi.org/pypi/safety/json | jq -r '.info.version'
```

Install

```
pip install safety
```

Verify installation

```
safety --version
# Expected: safety, version x.y.z
```

Offline operation

safety requires internet for DB queries. For offline environments, use trivy as the primary SCA scanner.

Functional test

```
cat > /tmp/req_vuln.txt << 'EOF'
Pillow==9.0.0
EOF

safety check -r /tmp/req_vuln.txt
# Expected: vulnerabilities found, exit code 64

cat > /tmp/req_clean.txt << 'EOF'
Pillow==10.3.0
EOF

safety check -r /tmp/req_clean.txt
# Expected: No known security vulnerabilities found, exit code 0

rm /tmp/req_vuln.txt /tmp/req_clean.txt
```

6.9 npm + npm audit — Node.js CVE (Layer 3)

What it does: built into npm — audits `package-lock.json` against the npm security advisory registry for known CVEs in Node.js dependencies.

Find the latest version

```
npm --version
node --version
# npm is bundled with Node.js; update both together
```

Install (via NodeSource — LTS)

```
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Verify installation

```
node --version    # Expected: v20.x.x or v22.x.x
npm  --version    # Expected: 10.x.x or later
npm audit --version 2>/dev/null || echo "npm audit is built-in"
```

Offline operation

npm audit requires internet to query the npm advisory registry. No offline mode is available.

Functional test

```
mkdir /tmp/npm-test && cd /tmp/npm-test

# Create a package.json with a known vulnerable package
cat > package.json << 'EOF'
{
  "name": "test",
  "version": "1.0.0",
  "dependencies": {
    "lodash": "4.17.4"
  }
}
EOF

npm install --package-lock-only 2>/dev/null
npm audit --json | jq '.metadata.vulnerabilities'
# Expected: object with non-zero high/critical counts

cd / && rm -rf /tmp/npm-test
```

6.10 Bandit — Python SAST (Layer 5)

What it does: Python-specific static analyser with 100+ plugins detecting SQL injection, shell injection, hardcoded passwords, weak crypto, insecure deserialization, and more.

Find the latest version

```
curl -s https://pypi.org/pypi/bandit/json | jq -r '.info.version'
```

Install

```
pip install bandit
```

Verify installation

```
bandit --version
# Expected: bandit x.y.z  (Python x.y.z)
```

Offline operation

Bandit has no external database. Works fully offline.

Functional test

```
cat > /tmp/test_bandit.py << 'EOF'
```

```
import subprocess
import hashlib

# B602 - subprocess with shell=True
def run(cmd):
    subprocess.call(cmd, shell=True)

# B324 - use of weak hash
def hash_password(pwd):
    return hashlib.md5(pwd.encode()).hexdigest()
EOF

bandit -ll /tmp/test_bandit.py
# Expected: Issue: [B602] ... [B324] ... exit code 1

cat > /tmp/test_bandit_clean.py << 'EOF'
import subprocess
import hashlib

def run(cmd_list):
    subprocess.call(cmd_list)

def hash_password(pwd):
    return hashlib.sha256(pwd.encode()).hexdigest()
EOF

bandit -ll /tmp/test_bandit_clean.py
# Expected: No issues identified, exit code 0

rm /tmp/test_bandit.py /tmp/test_bandit_clean.py
```

6.11 ShellCheck — Shell Script SAST (Layer 5)

What it does: static analyser for bash/sh/dash/ksh. Detects quoting errors, command injection risks, undefined variables, deprecated syntax, and unsafe patterns.

Find the latest version

```
apt-cache policy shellcheck | grep Candidate
# or latest binary from GitHub:
curl -s https://api.github.com/repos/koalaman/shellcheck/releases/latest \
    | jq -r '.tag_name'
```

Install (apt)

```
sudo apt-get install -y shellcheck
```

Install (latest binary from GitHub)

```
VERSION=$(curl -s https://api.github.com/repos/koalaman/shellcheck/releases/latest \
    | jq -r '.tag_name')
curl -sSL "https://github.com/koalaman/shellcheck/releases/download/${VERSION}/shellcheck-${VERSION}.linux.x86_64.tar.xz" \
    | tar -xJ --strip-components=1 -C /tmp
sudo mv /tmp/shellcheck /usr/local/bin/shellcheck
```

Verify installation

```
shellcheck --version
```

```
# Expected: version: x.y.z
```

Offline operation

ShellCheck has no external database. Works fully offline.

Functional test

```
cat > /tmp/test.sh << 'EOF'
#!/bin/bash
filename=$1
if [ $filename == "admin" ]; then
  echo "Hello $filename"
fi
rm -rf /$filename
EOF

shellcheck --severity=warning /tmp/test.sh
# Expected: SC2086 (unquoted variable), SC2115 (unsafe rm), exit code 1

cat > /tmp/test_clean.sh << 'EOF'
#!/bin/bash
filename="$1"
if [[ "$filename" == "admin" ]]; then
  echo "Hello $filename"
fi
EOF

shellcheck --severity=warning /tmp/test_clean.sh
# Expected: exit code 0

rm /tmp/test.sh /tmp/test_clean.sh
```

6.12 cppcheck — C/C++ SAST (Layer 5)

What it does: static analyser for C and C++ detecting buffer overflows, memory leaks, null-pointer dereferences, use-after-free, and undefined behaviour without compiling.

Find the latest version

```
apt-cache policy cppcheck | grep Candidate
# or from GitHub:
curl -s https://api.github.com/repos/danmar/cppcheck/releases/latest \
    | jq -r '.tag_name'
```

Install (apt)

```
sudo apt-get install -y cppcheck
```

Verify installation

```
cppcheck --version
# Expected: Cppcheck x.y
```

Offline operation

cppcheck has no external database. Works fully offline.

Functional test

```
cat > /tmp/test.cpp << 'EOF'
#include <string.h>
void test() {
    char buf[10];
    strcpy(buf, "Hello, this string is too long and will overflow the buffer!");
    int* p = new int(42);
    delete p;
    *p = 100;  // use after free
}
EOF

cppcheck --enable=warning,security --error-exitcode=1 /tmp/test.cpp
# Expected: errors for buffer overflow and use-after-free, exit code 1

cat > /tmp/test_clean.cpp << 'EOF'
#include <cstring>
void test() {
    char buf[100];
    strncpy(buf, "Hello", sizeof(buf) - 1);
    buf[sizeof(buf) - 1] = '\0';
}
EOF

cppcheck --enable=warning,security --error-exitcode=1 /tmp/test_clean.cpp
# Expected: no errors, exit code 0

rm /tmp/test.cpp /tmp/test_clean.cpp
```

6.13 hadolint — Dockerfile Linter (Layer 5)

What it does: enforces Dockerfile best practices and detects security misconfigurations: running as root, `:latest` tags, baked-in secrets, ADD vs COPY, shell injection.

Find the latest version

```
curl -s https://api.github.com/repos/hadolint/hadolint/releases/latest \
    | jq -r '.tag_name'
```

Install (binary)

```
VERSION=$(curl -s https://api.github.com/repos/hadolint/hadolint/releases/latest \
    | jq -r '.tag_name')
curl -sSL "https://github.com/hadolint/hadolint/releases/download/${VERSION}/hadolint-Linux-x86_64" \
    -o /usr/local/bin/hadolint
sudo chmod +x /usr/local/bin/hadolint
```

Verify installation

```
hadolint --version
# Expected: Haskell Dockerfile Linter x.y.z
```

Offline operation

hadolint has no external database. Works fully offline.

Functional test

```
cat > /tmp/Dockerfile_bad << 'EOF'
```

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y curl
ADD https://example.com/script.sh /tmp/
ENV PASSWORD=mysecretpassword
RUN /tmp/script.sh
EOF

hadolint /tmp/Dockerfile_bad
# Expected: DL3007 (latest tag), DL3009 (delete apt cache), SC2046, etc.

cat > /tmp/Dockerfile_good << 'EOF'
FROM ubuntu:22.04
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
COPY script.sh /tmp/script.sh
USER nobody
EOF

hadolint /tmp/Dockerfile_good
# Expected: no output or only style suggestions, exit code 0

rm /tmp/Dockerfile_bad /tmp/Dockerfile_good
```

6.14 checkov — Terraform / IaC SAST (Layer 5)

What it does: static analysis for Terraform, CloudFormation, Kubernetes YAML, Ansible and ARM templates. Maps findings to CIS Benchmarks, NIST, SOC2, and OWASP. 1000+ built-in checks.

Find the latest version

```
curl -s https://pypi.org/pypi/checkov/json | jq -r '.info.version'
```

Install

```
pip install checkov
```

Verify installation

```
checkov --version
# Expected: x.y.z
```

Offline operation

checkov uses local checks only — works fully offline. No external DB required.

Functional test

```
mkdir /tmp/tf-test

cat > /tmp/tf-test/main.tf << 'EOF'
resource "aws_s3_bucket" "public_bucket" {
  bucket = "my-public-bucket"
  acl    = "public-read"
}

resource "aws_security_group" "open" {
  ingress {
```

```
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
EOF

checkov -d /tmp/tf-test --framework terraform --compact
# Expected: FAILED checks (public S3 bucket, open SSH to world)

cat > /tmp/tf-test/main_secure.tf << 'EOF'
resource "aws_s3_bucket" "private_bucket" {
  bucket = "my-private-bucket"
}
resource "aws_s3_bucket_acl" "private" {
  bucket = aws_s3_bucket.private_bucket.id
  acl    = "private"
}
EOF

checkov -f /tmp/tf-test/main_secure.tf --framework terraform --compact
# Expected: all checks passed or significantly fewer failures

rm -rf /tmp/tf-test
```

6.15 ScanCode Toolkit — Licence & Copyright (Layer 6)

What it does: full-text licence detection using 30 000+ licence texts (SPDX), copyright notice extraction, package manifest detection, and CVE lookup in detected packages. Produces JSON/SPDX/CycloneDX reports.

Find the latest version

```
curl -s https://pypi.org/pypi/scancode-toolkit/json | jq -r '.info.version'
```

Install

```
pip install scancode-toolkit
```

> ScanCode is a large package (~500 MB installed). Installation may take 5–10 minutes.

Verify installation

```
scancode --version
# Expected: ScanCode version x.y.z
```

Offline operation

ScanCode works **fully offline** — all licence texts and detection logic are bundled in the package. No external DB or internet required at scan time.

Functional test

```
mkdir /tmp/sc-test

# File with a GPL licence header
```

```
cat > /tmp/sc-test/app.py << 'EOF'
# This program is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.
# Copyright (C) 2024 Example Corp.
import os

def main():
    print("hello")
EOF

# File with a permissive MIT licence
cat > /tmp/sc-test/lib.py << 'EOF'
# MIT License
# Copyright (c) 2024 Example Corp
# Permission is hereby granted, free of charge, to any person obtaining a copy...
def helper():
    pass
EOF

scancode --license --copyright \
    --json-pp /tmp/sc-report.json \
    --quiet \
    /tmp/sc-test

# Check detections
jq '.files[] | {path: .path, licenses: [.license_detections[].matches[].spdx_license_expression]}' \
    /tmp/sc-report.json

# Expected: GPL-3.0-or-later detected in app.py, MIT in lib.py

rm -rf /tmp/sc-test /tmp/sc-report.json
```

7. Pipeline Scripts Installation

7.1 Clone from GitHub

```
git clone https://github.com/gaillotte/claude.git
cd claude
git checkout claude/vigilant-carson-f8twy0
```

7.2 Install Python dependencies for report generation

```
pip install openpyxl reportlab
```

7.3 Make scripts executable

```
chmod +x ai_transit.sh fetch_repo.sh scan_pipeline.sh
```

7.4 Verify scripts

```
bash -n ai_transit.sh    && echo "ai_transit.sh: syntax OK"
```

```
bash -n fetch_repo.sh    && echo "fetch_repo.sh: syntax OK"
bash -n scan_pipeline.sh && echo "scan_pipeline.sh: syntax OK"
python3 -c "import ast; ast.parse(open('generate_excel_report.py').read())" \
    && echo "generate_excel_report.py: syntax OK"
python3 -c "import ast; ast.parse(open('selfcheck.py').read())" \
    && echo "selfcheck.py: syntax OK"
```

7.5 Run the test suite

This is the fastest way to confirm the installation is sound. It needs no scanning tools — see §17.

```
./tests/run_tests.sh
# Expected: ✔ 740/740 passed
```

7.6 Generate the integrity manifest

```
python3 selfcheck.py --write-manifest
```

Do this once the bundle is in its final location. See §14.2 for why the manifest is generated at install time rather than shipped in version control.

8. Directory Structure Setup

```
sudo mkdir -p /opt/ai-transit/{fetch,quarantine,approved,reports,logs,yara-rules}
sudo chmod 700 /opt/ai-transit/quarantine
sudo chown -R $USER:$USER /opt/ai-transit
mkdir -p Good
```

Copy your YARA rules to `/opt/ai-transit/yara-rules/` (see §6.4 for a minimal test rule).

9. Environment Variables & Flags

9.1 Command-line flags

```
./ai_transit.sh [FLAGS] <repo_url_or_path> [branch]

  --quiet               Suppress all output except the final PASS/FAIL verdict (CI mode)
  --verbose             Full debug output including per-file scan details
  --min-severity LEVEL  Severity threshold: low | medium | high (default) | critical
                        Findings below the threshold are logged as WARN, not FAIL
  --since COMMIT        Diff mode: only scan files changed since this commit SHA
  --report-only         Always exit 0 even on FAIL (observation / audit mode).
                        Also leaves the fetched repo in place instead of
                        quarantining it, so nothing is moved or deleted.
  --no-zip              Skip creation of the approved ZIP archive
  --no-excel            Skip generation of the Excel report
  --offline             Air-gapped mode: no scanner attempts a network call.
                        Uses locally staged rules and databases, and reports any
                        layer that could not run. See section 10.
```

`--no-zip --no-excel` is the usual combination for CI, where the JSON and HTML reports are consumed by another job and the archive would only be discarded.

9.2 Environment variables

Variable	Default	Description
`WORK_DIR`	`/opt/ai-transit`	Root working directory
`OUTPUT_DIR`	`<script_dir>/Good`	Output directory for approved ZIPs
`GITHUB_TOKEN`	*(unset)*	Personal access token for private GitHub repos
`MAX_SIZE_MB`	`500`	Repository size limit in MB
`MIN_SEVERITY`	`high`	Minimum severity to block (`low\
`VERBOSITY`	`normal`	Log verbosity passed to scanner (`quiet\
`SINCE_COMMIT`	*(unset)*	Diff mode commit SHA (same as `--since`)
`OFFLINE`	`false`	Air-gapped mode (same as `--offline`)
`OFFLINE_CACHE`	`$WORK_DIR/offline-cache`	Root of staged offline assets
`SEMGREP_RULES_DIR`	`$OFFLINE_CACHE/semgrep-rules`	Exported Semgrep rulesets
`TRIVY_CACHE_DIR`	`$OFFLINE_CACHE/trivy-db`	trivy vulnerability database
`CLAMAV_DB_DIR`	`$OFFLINE_CACHE/clamav`	ClamAV signature files

Add persistent values to `~/.bashrc`:

```
export WORK_DIR="/opt/ai-transit"
export OUTPUT_DIR="/opt/ai-transit/approved"
```

Or pass inline:

```
WORK_DIR=/data/ai-transit ./ai_transit.sh https://github.com/org/repo
```

9.3 Per-repo exclusions

Place either of these files in the **root of the scanned repository** (not the pipeline directory):

File	Format	Effect
`.transitignore`	gitignore-style patterns	Files matching patterns are excluded from all scan layers
`.transit-allow.json`	JSON array of `{rule, path, reason}`	Matching FAIL findings are downgraded to WARN

`.transit-allow.json` **example:**

```
[
  {
    "rule": "CWE-798",
    "path": "tests/fixtures/dummy_key.py",
    "reason": "Test fixture — not a real credential"
  },
  {
    "rule": "binary",
    "path": "assets/logo.png",
```

```
    "reason": "Approved binary asset"
  }
]
```

10. Full Installation Verification

Save this as `verify_install.sh` and run it after completing all sections above:

```
#!/usr/bin/env bash
# verify_install.sh — checks all required and optional tools

PASS=0; WARN=0; FAIL=0

check()   {
    local name="$1" cmd="$2"
    if command -v "$cmd" &>/dev/null; then
        echo -e "\033[32m[OK]\033[0m    $name"
        (( PASS++ ))
    else
        echo -e "\033[33m[WARN]\033[0m  $name — optional, not installed"
        (( WARN++ ))
    fi
}

require() {
    local name="$1" cmd="$2"
    if command -v "$cmd" &>/dev/null; then
        echo -e "\033[32m[OK]\033[0m    $name"
        (( PASS++ ))
    else
        echo -e "\033[31m[FAIL]\033[0m  $name — REQUIRED, not installed"
        (( FAIL++ ))
    fi
}

echo "=== Core system ==="
require "bash"        bash
require "git"         git
require "python3"     python3
require "pip"         pip
require "curl"        curl
require "jq"          jq
require "zip"         zip
require "sha256sum"   sha256sum
require "file"        file

echo ""
echo "=== Python report packages ==="
python3 -c "import openpyxl"  2>/dev/null \
    && { echo -e "\033[32m[OK]\033[0m    openpyxl"; (( PASS++ )); } \
    || { echo -e "\033[31m[FAIL]\033[0m  openpyxl — pip install openpyxl"; (( FAIL++ )); }
python3 -c "import reportlab" 2>/dev/null \
    && { echo -e "\033[32m[OK]\033[0m    reportlab"; (( PASS++ )); } \
    || { echo -e "\033[31m[FAIL]\033[0m  reportlab — pip install reportlab"; (( FAIL++ )); }

echo ""
echo "=== Layer 1 — Secrets & AV ==="
check "gitleaks"        gitleaks
check "detect-secrets"  detect-secrets
check "ClamAV"          clamscan
check "YARA"            yara

echo ""
echo "=== Layer 2 — OWASP/CWE ==="
```

```
check "Semgrep"         semgrep

echo ""
echo "=== Layer 3 — SCA/CVE ==="
check "trivy"           trivy
check "pip-audit"       pip-audit
check "safety"          safety
check "npm"             npm

echo ""
echo "=== Layer 5 — SAST ==="
check "Bandit"          bandit
check "ShellCheck"      shellcheck
check "cppcheck"        cppcheck
check "hadolint"        hadolint
check "checkov"         checkov

echo ""
echo "=== Layer 6 — Licence ==="
check "scancode"        scancode

echo ""
echo "=== Summary ==="
echo -e "  \033[32mOK: $PASS\033[0m  |  \033[33mOptional missing: $WARN\033[0m  |  \033[31mRequired missing: $FAIL\033[0m"
[[ $FAIL -eq 0 ]] \
    && echo -e "\033[32mInstallation COMPLETE — pipeline ready\033[0m" \
    || echo -e "\033[31mInstallation INCOMPLETE — fix FAIL items above\033[0m"
```

```
chmod +x verify_install.sh && ./verify_install.sh
```

10.1 Final step — verify the install works offline

Tool presence is not the same as tool *readiness*. Once the offline cache is staged (§11.4), disconnect the machine and run:

```
./verify_offline_install.sh
```

It checks each tool individually before running the pipeline, because the pipeline degrades quietly: a tool that cannot work offline yields an empty result, which looks exactly like a clean one. Full walkthrough: §2, Stage C.

11. Air-Gapped Operation

`scan_pipeline.sh` has **no online mode**: the scan layers always run disconnected, on locally staged data. `ai_transit.sh --offline` therefore only decides whether a remote URL may be fetched (it is refused). This section is the reference for what each tool needs.

11.1 Why the scan is offline-only

Several scanners reach the network at **scan** time, not just at install time. Allowed to go online on an isolated host, those calls are still attempted. They do not crash the run — each is individually guarded — but they block on DNS and TCP timeouts and then return nothing.

The consequence is worse than slowness. Layer 2 (OWASP/CWE) and Layer 3 (CVE) contribute no findings, while the report still presents them as having run. The verdict can be **PASS on a repository that was never meaningfully scanned**.

Offline operation changes that: each tool is pointed at locally staged data and given the flags that suppress update attempts and telemetry, and any layer that genuinely cannot run is recorded as an

explicit `OFFLINE:` warning naming what did not execute. A silent gap becomes a stated one. A caller that sets `OFFLINE=false` gets a warning and an offline scan.

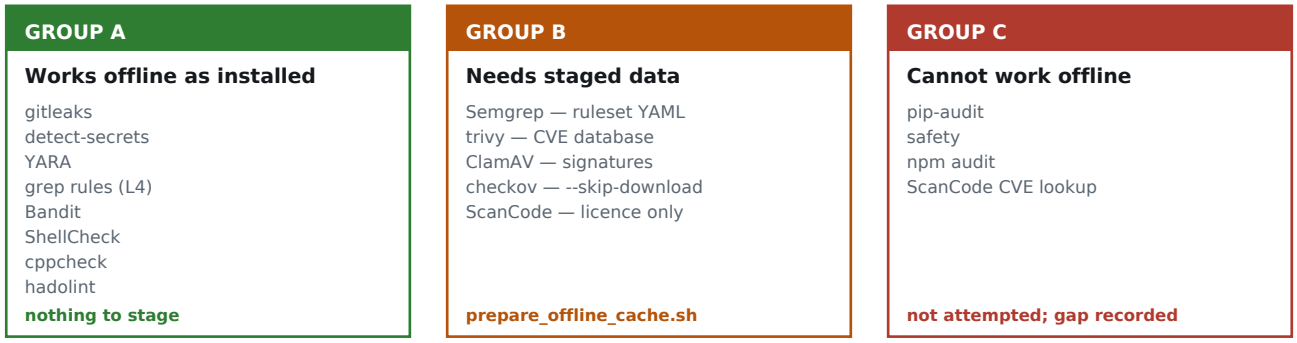
Layers 3 and 6 are switched off by default. Dependency CVE scanning (layer 3) and ScanCode licences (layer 6) are skipped unless `RUN_SCA=true` / `RUN_SCANCODE=true` is set. The report records each as `skipped:disabled`, so `coverage_complete` is false until they are enabled: a layer that did not run must not read as one that found nothing.

```
export OFFLINE_CACHE=/opt/ai-transit/offline-cache
./ai_transit.sh --offline /path/to/repo
```

> **For the step-by-step operating procedure** — building and transferring the cache, verifying it on arrival, and the acceptance gate that rejects a scan whose verdict rests on layers that never ran — see **OFFLINE_RUNBOOK.md**.

Remote URLs are refused in this mode — copy the repository to the host and pass its path.

11.2 Per-tool reference



> **Offline, the staged trivy database is the ONLY dependency-CVE coverage there is.**
> Connected, four tools overlap on that job and losing one is survivable. Offline there is no redundancy.

Figure 4 — offline capability of each tool, and the one gap with no redundancy.

Group A — works offline with no preparation. Rules are compiled into the tool; nothing to stage, no flags required.

Tool	Layer	Notes
gitleaks	L1	Detection rules embedded in the binary
detect-secrets	L1	Entropy and regex rules embedded
YARA	L1	Reads your own `.yar` files from `$WORK_DIR/yara-rules/`
grep built-ins	L4	CWE-798/22/918/327/338 patterns, entirely local
PDF scanner (`scan_pdf`)	L5	Built in; needs only `python3` and its standard library — no signatures, no rules to stage, no network at any point
Makefile scanner (`scan_makefile`)	L5	Built in; pure pattern matching, nothing to stage
HTML scanner (`scan_html`)	L5	Built in; pure pattern matching, nothing to stage
CMake scanner (`scan_cmake`)	L5	Built in; pure pattern matching, nothing to stage

Tool	Layer	Notes
Windows batch (`scan_bat`)	L5	Built in; pure pattern matching, nothing to stage
Java archives (`scan_jar`)	L5	Built in; needs `python3` (standard library only), reads the archive in place — no extraction, no network, nothing to stage
Office documents (`scan_office`)	L5	Built in; needs `python3` (standard library only), reads the document in place — no extraction, no network, nothing to stage
PowerShell (`scan_powershell`, `scan_powershell_xml`)	L5	Built in; pure pattern matching, nothing to stage
Known credential formats (Layer 4, every file)	L4	Built in; pure pattern matching, nothing to stage
`setup.py` / `conanfile.py` (`scan_setup_py`)	L5	Built in; pure pattern matching, nothing to stage
Gradle (`scan_gradle`)	L5	Built in; pure pattern matching, nothing to stage
Package manifests (`scan_package_json`, `scan_requirements`)	L5	Built in; needs `python3` (standard library) to parse JSON, otherwise falls back to patterns with a WARN
XML / Maven / Ant (`scan_xml`)	L5	Built in; pattern matching, plus `python3` to strip comments — falls back to scanning the raw file without it
Bandit	L5	Local Python AST analysis
ShellCheck	L5	Local
cppcheck	L5	Local
hadolint	L5	Local

Group B — works offline once data is staged. These are the ones that make air-gapped operation worth configuring properly.

Tool	Layer	Must be staged	Flags applied by `--offline`
Semgrep	L2	One YAML per ruleset in `$SEMGREP_RULES_DIR`	`--config <file>` instead of `p/<name>`, plus `--metrics=off`
trivy	L3	Vulnerability DB in `$TRIVY_CACHE_DIR`	`--skip-db-update --skip-java-db-update --offline-scan --cache-dir`
ClamAV	L1	`*.cvd` / `*.cld` in `$CLAMAV_DB_DIR`	`--database=<dir>`
checkov	L5	Nothing	`--skip-download` (stops remote provider-schema fetches)
ScanCode	L6	Nothing for licence/copyright	`--vulnerability` is **dropped**; licence and copyright detection are local and still run

Group C — no offline mode exists. These query a remote advisory service by design and cannot be staged. The scan is always offline, so they are not attempted (layers 3 and 6 are in any case off unless `RUN_SCA=true` / `RUN_SCANCODE=true`), and each emits an `OFFLINE:` warning when it would have run, so the gap is on the record.

Tool	Layer	Why	Mitigation
pip-audit	L3	Resolves advisories from PyPI/OSV	Python CVEs covered by the staged trivy DB

Tool	Layer	Why	Mitigation
safety	L3	Queries the Safety DB service	As above
npm audit	L3	Queries the npm registry endpoint	JS CVEs covered by the staged trivy DB
ScanCode `--vulnerability`	L6	Queries VulnerableCode	Package CVEs covered by the staged trivy DB
GitHub API size check	fetch	Needs `api.github.com`	Not applicable: `--offline` accepts local paths only

The single practical consequence: **trivy is the only dependency-CVE coverage you have offline**, and only when layer 3 is enabled (`RUN_SCA=true`). If its database is not staged, Layer 3 contributes nothing.

11.2.1 Build files — `Makefile`, `.mk`, `.mak`

`scan_makefile` covers `.mk`, `.mak` and `.make` by extension, and `Makefile`, `makefile` and `GNUmakefile` by name. It is pure pattern matching, so it needs nothing staged and behaves identically offline.

It exists because a Makefile is more dangerous than it looks. Recipe lines are shell, and `$(shell …)` is evaluated when make **reads** the file — before any target runs. `make -n`, or `make` with a mistyped target name, is enough to fire it, so a hostile Makefile does not need to be built to act. Separately, Makefile assignments carry no quotes, which means `PASSWORD = hunter2` slips past the universal credential rule (that one requires a quote after the `=`).

FAIL: a download piped into a shell · a `$(shell …)` that fetches · a base64-decoded payload executed · a reverse shell · a hardcoded credential · TLS verification disabled · a setuid/setgid bit · cron or `authorized_keys` persistence · `rm -rf` of `/` or `$HOME`.

WARN: `sudo` in a recipe · `chmod 777` · a plaintext `http://` download · clone-and-build from a remote repository.

The rules are deliberately narrow, because in a Makefile the dangerous commands look almost exactly like the ordinary ones:

Ordinary — never reported	Reported
`rm -rf $(BUILD)` in a clean target	`rm -rf /`
`curl -fsSL url -o file`	`` `curl -fsSL url \ ``
`$(shell git rev-parse --short HEAD)`	`$(shell curl -s url)`
`API_KEY ?= / TOKEN = $(CI_TOKEN)`	`PASSWORD = hunter2`
`install -m 0755 prog /usr/bin`	`install -m 4755 prog /usr/bin`

Verify both directions:

```
WORK_DIR=/tmp/mkcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected: evil.mk FAIL (all ten rules), Makefile FAIL (credential only),
#           safe.mk PASS with no findings at all.
```

As a false-positive check, 88 real Makefiles collected from a Debian host — dpkg, java, vim and postgresql build fragments — produce **zero findings**.

11.2.2 CMake — `.cmake` and `CMakeLists.txt`

`scan_cmake` is pure pattern matching, so nothing needs staging and it behaves identically offline. It covers `.cmake` by extension and `CMakeLists.txt` by name — the latter matters, because `CMakeLists.txt` ends in `.txt` and would otherwise be scanned as a plain text document.

Two properties of CMake drive the rules. `file(DOWNLOAD …)` and `execute_process(…)` run at **configure** time, so `cmake .` alone is enough to fetch and run something — there is no build step to decline. And `set(NAME "value")` is a function call rather than an assignment, so a credential written that way is invisible to the universal credential rule, which expects `key = "value"`.

FAIL: a credential in `set()` · a download over plaintext `http://` · `TLS_VERIFY OFF` · a download piped into a shell · hardening removed (`-fno-stack-protector`, `_FORTIFY_SOURCE=0`, `execstack`) · `file(WRITE …)` into `/etc`, `~/.ssh` or cron.

WARN: a build step that fetches from the network · `COMMAND sh -c` · `install(CODE|SCRIPT)` · a `GIT_TAG` pinned to a moving branch.

The distinctions that keep it usable, each one drawn from the 1116-file corpus:

Ordinary — never reported	Reported
`execute_process(COMMAND git rev-parse …)` — in 106 of 1116 files	`execute_process(COMMAND sh -c "curl … \`
`set_package_properties(… URL "http://xmlsoft.org/")` — a homepage	`file(DOWNLOAD http://host/x.tar.gz …)`
`file(DOWNLOAD https://… EXPECTED_HASH SHA256=…)`	the same without TLS or without a hash
`set(DROP_SITE_PASSWORD "${CTEST_DROP_SITE_PASSWORD}")`	`set(DEPLOY_API_KEY "sk_live_…")`
`set(… RPATH_ORIGIN_TOKEN "\$ORIGIN")` — a name containing TOKEN	—
`-fstack-protector-strong`	`-fno-stack-protector`

```
WORK_DIR=/tmp/cmakecheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected: evil.cmake FAIL, CMakeLists.txt FAIL (credential only),
#           safe.cmake PASS — it carries the near-miss of every rule.
```

Across the 1116 real files **1 fails**, and that one is a plaintext archive URL written inside CMake's own `ExternalProject` documentation.

11.2.3 HTML — `.html`, `.htm`, `.xhtml`

`scan_html` is pure pattern matching, so it needs nothing staged and behaves identically offline.

Inline `<script>` is ordinary, so the rules target only constructs with no innocent explanation. Which those are was settled by counting each candidate rule against 388 real HTML files — Go tool bundles, Playwright reports, gcloud and Python package documentation — and the counts are recorded beside each rule in `scan_pipeline.sh`.

FAIL: `eval()` · a percent-encoded payload in `unescape()` · `document.write()` fed from a decoder · a script loaded over `http://` · an iframe with no visible size · `<base href="http://">` · a form posting over `http://` · a credential in a `name`/`value` attribute pair or a `<meta content>` · a cookie read wired straight to a network sink.

WARN: `innerHTML` and friends · `new Function()` · a bare `atob()` · a protocol-relative `<script src="//…">` · an external meta refresh · a `javascript:` URI.

The distinctions that keep it usable:

Ordinary — never reported	Reported
`unescape(encodeURIComponent(s))`	`unescape('%u9090%u9090')`
`<script src="https://cdn/lib.js">`	`<script src="http://cdn/lib.js">`
`<iframe src="…" width="600" height="400">`	`<iframe … style="display:none">`
`<form action="/session">`	`<form action="http://host/harvest">`
a cookie read, and a `fetch()` elsewhere	both inside one statement

```
WORK_DIR=/tmp/htmlcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected: evil.html FAIL (ten rules), tracker.htm FAIL (hidden iframe),
#           safe.html PASS — it carries the near-miss of almost every rule.
```

Across those 388 real files only **two** are failed by an `scan_html` rule, both the bundled Go trace viewer, which genuinely calls `eval()`. Vendoring a file like that is what `.transit-allow.json` is for:

```
[{"rule": "CWE-95", "path": "third_party/trace_viewer_full.html",
  "reason": "vendored Go trace viewer; eval is inherent to the tool"}]
```

> **NOTE**
> .** On that same corpus the universal Layer 4 traversal rule used to fail 254 of the 388 files, matching ordinary relative links such as `href="../commands"`. It has since been narrowed to require an encoded `..`, a sensitive target, or traversal composed with input, and now flags none of them. After that change, and the key-body requirement described in §11.2.5, 2 of the 388 fail, both the Go trace viewer's genuine `eval()`.

11.2.4 PDF scanning — no staging, and why that matters

`scan_pdf` is the one content-inspection layer with no perishable data behind it. It needs `python3` and its standard library and nothing else: no signature database, no rule bundle, no registry lookup. On an air-gapped host it is exactly as capable as on a connected one, which is not true of Layers 2, 3 or 6.

That is worth knowing because a PDF is a container, not a flat document. The format can run JavaScript the moment a file is opened (`/OpenAction`, `/AA`), start an external program (`/Launch`) and carry an embedded executable (`/EmbeddedFile`). Those are **FAIL** conditions. Things the scanner cannot see into — `/Encrypt`, `/XFA`, a stream behind a filter it cannot decode — are **WARN**, named individually, because an uninspected stream is not a clean one.

Object dictionaries need not be in plain text: from PDF 1.5 they may live inside an `/ObjStm`, so the scanner decodes each stream's declared filter chain (`FlateDecode`, `LZWDecode`, `ASCIIHexDecode`, `ASCII85Decode`, `RunLengthDecode`) before any rule runs, with a 64 MB ceiling against decompression bombs. Chains are ordinary in practice — ReportLab, which builds this very guide, writes `/Filter [ /ASCII85Decode /FlateDecode ]`.

Verify it the way the test suite does:

```
# Build the fixtures, then scan them
python3 tests/fixtures/make_pdf_fixtures.py
WORK_DIR=/tmp/pdfcheck bash scan_pipeline.sh tests/fixtures/rules --quiet

# Expected: verdict FAIL.
#   pdf_openaction_js.pdf     FAIL  CWE-94:PDF_javascript_embedded, PDF_auto_execute…
#   pdf_compressed_js.pdf     FAIL  payload was only inside a FlateDecode /ObjStm
```

```
#   pdf_obfuscated_js.pdf     FAIL  names were written /J#61vaScript
#   pdf_ascii85_js.pdf        FAIL  payload behind ASCII85Decode
#   pdf_lzw_js.pdf            FAIL  payload behind LZWDecode
#   pdf_paren_swallow_js.pdf  FAIL  stray "(" did not hide the dictionary after it
#   benign_doc.pdf            PASS  ordinary document — must not be flagged
```

The last two lines are the ones that matter on an upgrade. `benign_doc.pdf` embeds a subset font, so it carries `/AAAAAA+DejaVuSans` tags; an `/AA` rule without a right-hand anchor reads those as an AdditionalActions key and fails ordinary documentation — including this guide. And because this guide contains the words "Python/JavaScript dependency CVE", which a content stream renders as `(Python/JavaScript dependency CVE) Tj`, key rules read a view of the file with string literals blanked out. Anchoring alone cannot separate those cases: `/S/JavaScript` is valid PDF, because names self-delimit.

Acceptance test after any change to `scan_pdf` — the five PDFs this project ships are the realistic negative set:

```
mkdir -p /tmp/ownpdfs && cp *.pdf /tmp/ownpdfs/
WORK_DIR=/tmp/pdfcheck bash scan_pipeline.sh /tmp/ownpdfs --quiet
# Expected: PASS, with no findings against any of the five.
```

Not covered: an encrypted PDF is reported but not decrypted, so its streams go uninspected. Treat a `pdf:encrypted_content_could_not_be_inspected` WARN as a file needing manual review, not a clean result.

11.2.5 Traversal and key material — the rules that run on every file

Most rules are chosen by file type. Two are not: path traversal and private key material are checked on **every** file by the Layer 4 universal patterns. That reach is why both were narrowed, and why both are worth understanding before you read a report.

Path traversal. A literal `../` is a relative path, not an attack. The original rule matched any occurrence, which failed 254 of 388 real HTML files on links like `href="../commands"` and 27 of 88 real Makefiles on `-I../include` — quarantining repositories for containing documentation. Traversal is now reported only in the three contexts that indicate it:

Finding	Fires on
`CWE-22:path_traversal_encoded`	`%2e%2e, ..%2f, %252e%252e`
`CWE-22:path_traversal_to_sensitive_path`	`../../etc/passwd, ..\\windows\\system32, ../.ssh/, id_rsa`
`CWE-22:path_traversal_composed_with_input`	`"../" + name, f"{dir}/../{f}", $_GET['p'] . '../'`

So these are **not** reported, and should not be:

```
CFLAGS += -I../include                    VPATH = ../src
<a href="../commands/index.html">…</a>    from ..models import User
require('../package.json')                #include "../common/defs.h"
```

Key material. `CWE-321:hardcoded_private_key` requires a real PEM header, `-----BEGIN … PRIVATE KEY-----`. The original rule matched the bare words `PRIVATE KEY`, `CERTIFICATE` and `BEGIN RSA`, so a document that merely *discussed* keys failed the repository.

A public certificate is reported as `crypto:public_certificate_embedded`, a **WARN** rather than a FAIL: a certificate is the public half of the pair and is published by design, so it is not a leaked secret.

Verify both directions:

```
WORK_DIR=/tmp/univcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected: traversal.txt and private_key.txt FAIL;
#           safe_relative_paths.txt and safe_key_prose.txt do not
#           (safe_key_prose.txt carries only the public-certificate WARN).
```

A PEM header on its own is not a key, so `CWE-321` also requires a base64 body right after it (60 or more characters within five lines). Two things carry a header and no key: the shared-mime-info database, whose `magic="-----BEGIN PGP PRIVATE KEY BLOCK-----"` describes how to *recognise* a key file, and documentation that shows the format with a placeholder, such as npm's `key="-----BEGIN PRIVATE KEY-----\nXXXX\nXXXX\n-----END PRIVATE KEY-----"`. A key held on one line as a JSON string, and a PGP block with armor headers, are both still caught.

After narrowing, **2 of the 388 real HTML files fail**, both the bundled Go trace viewer's genuine `eval()`. The 88-file Makefile, 1116-file CMake and 3362-file XML corpora have no key finding at all.

> An earlier revision of this section said three HTML files "embed a private key in an example TLS config". That was a misreading: all three are npm's configuration documentation with an `XXXX` placeholder body.

11.2.6 XML, Maven and Ant

`scan_xml` covers `.xml` and `.pom`. Maven's `pom.xml` and Ant's `build.xml` have no scanner of their own — they arrive here — so this is the whole of what a Maven or Ant repository receives. It is pure pattern matching and behaves identically offline.

XXE. The primitive is an entity that points at something, not a DOCTYPE. A bare `<!DOCTYPE` used to fail **736 of 3362** real XML files (22%), so a Maven or Ant repository was quarantined for containing one. The scanner now fails only an `<!ENTITY … SYSTEM` aimed at `file:`, `http(s):`, `php:`, `expect:` and similar, or an expansion bomb; a relative `SYSTEM` include or a `PUBLIC` DTD reference only warns. The declaration is matched with newlines folded, so splitting it across lines does not evade the rule, and commented-out XML is ignored.

Credentials are matched on the element name — `<password>`, `<token>`, `<secret>`, `<apiKey>`, `<privateKey>` — so an attribute called `tokens` in a checkstyle file no longer counts. These forms are correct and not reported:

Not reported	Reported
`<password>${env.REPO_PASSWORD}</password>`	`<token>9cbaf023786cd7a8b9e4f1d2c3a5b7e9</token>`
`<password>{3iRQQy…=}</password>` (Maven-encrypted)	
`<privateKey>/path/to/key</privateKey>`	
a `<password>` inside `<!-- … -->`	

Maven. Fails a `<repository>` over `http://`, and `exec-maven-plugin` or antrun running `sh`, `bash`, `cmd`, `powershell`, `curl` or `wget`. The repository test is scoped to the repository block on purpose: the default Maven archetype writes `<url>http://maven.apache.org</url>` as a project homepage, and an unscoped rule would fail every generated POM. `-SNAPSHOT` is deliberately not reported — 176 of 221 real poms use one.

Ant. Fails `<get>` over `http://`, `<exec>` of a shell or downloader, and an `<sshexec>` or `<scp>` with a literal `password=`. The Ant rules rest on only 22 real files, all of which measured zero; treat them as less calibrated.

```
WORK_DIR=/tmp/xmlcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected FAIL: xxe.xml, xxe_split.xml, xml_bomb.xml, credential.xml,
#                pom.xml, build.xml
# Expected PASS: safe_xml.xml, safe.pom, build-safe.xml
```

Across **3362 real XML files** — 221 of them `pom.xml` — none fails.

11.2.7 Package manifests — `package.json`, `composer.json`, `requirements.txt`

These are routed by name. Before, `package.json` and `composer.json` received a single credential check and `requirements.txt` was scanned as a text document, so the part of a manifest that matters — code that runs when dependencies are installed — was never examined. Pure pattern matching plus `python3` (standard library only) to parse the JSON; nothing to stage, identical offline.

Manifest	Fails on	Warns on
`package.json`	a `preinstall` / `install` / `postinstall` / `prepare` hook piping a download into a shell, or decoding base64 into one · a dependency on `http://`, `git://` or `git+http://`	an install-time hook being present · a script that fetches · a git or tarball dependency
`composer.json`	the same hooks over `post-install-cmd` and friends · `secure-http: false` · an `http://` repository	a script that fetches
`requirements*.txt`	an index or requirement over `http://` or `git://` · credentials embedded in a URL	`--trusted-host` · `--extra-index-url` · a VCS requirement not pinned to a commit

`--extra-index-url` warns because pip takes the highest version from **any** configured index, so a public package sharing a name with a private one wins.

The JSON is parsed rather than pattern-matched. A key can be written `"post\u0069nstall"`, which npm reads as `postinstall` and a text rule never sees. Invalid JSON is checked by pattern instead, with a WARN saying so.

How well-calibrated is this? Honestly, less than the other scanners. Of 1193 real `package.json` on the reference host, **none fail and 27 warn**; of 47 requirements files, none fail. But that corpus is mostly published packages under `node_modules` rather than application roots, so it says little about how often an AI-generated repository's own manifest carries an install hook. These rules rest on what the constructs mean more than on a measured rate.

```
WORK_DIR=/tmp/pkgcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected FAIL: pkg_evil, pkg_escaped, pkg_invalid (hook only), composer_evil, req_evil
# Expected PASS: pkg_plain, pkg_safe (WARN: install hook), composer_safe, req_safe (WARN x2)
```

Not covered: lockfiles (`package-lock.json`, `yarn.lock`, `go.sum`), which are generated integrity data; `go.mod`, which cannot execute anything; and `meson_options.txt`, which is declarative.

11.2.8 Gradle — `*.gradle`, `*.gradle.kts`, `gradle.properties`, `gradle-wrapper.properties`

`scan_gradle` is pure pattern matching, so nothing needs staging and it behaves identically offline. Matching is **by name**: `*.gradle.kts` ends in `.kts`, which would otherwise send it to the Kotlin *source* scanner — rules for `Runtime.exec` and weak hashes, and none of the build constructs below.

A Gradle script is a program, not a declarative file. It is evaluated in full at **configuration** time — on `gradle tasks`, on `--dry-run`, and automatically when an IDE imports the project — so a hostile top-level statement runs before any target is chosen, and `settings.gradle` runs before any project

exists.

File	Fails on	Warns on
*.gradle, *.gradle.kts	apply from: an http(s) URL · allowInsecureProtocol = true · a process started from a literal or list, a ProcessBuilder, Runtime.exec	a configuration-time remote fetch · exec / commandLine / javaexec
gradle.properties	a literal *.password / *.secret / *.apiKey · dependency.verification=off · -javaagent in org.gradle.jvmargs	a trustStore override
gradle-wrapper.properties	a plaintext distributionUrl	validateDistributionUrl=false

The rules are narrow because the ordinary forms look like the hostile ones. Measured against ~6290 real build scripts:

Ordinary — never reported	Reported
url = 'http://www.apache.org/licenses/…' in POM metadata (21 of 6249)	allowInsecureProtocol = true
query.execute(), a Gradle API call	"curl …".execute()
apply from: rootProject.file('common.gradle')	apply from: 'http://…'
signing.password= (blank)	signing.password=hunter2

That first row is why no rule keys on a URL scheme: Gradle 7+ refuses an http:// repository unless allowInsecureProtocol is set, so the flag is the reliable tell.

```
WORK_DIR=/tmp/gradlecheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected FAIL: gradle_evil/*, gradle_kts/build.gradle.kts, gradle_wrapper_evil/*
# Expected PASS: gradle_safe/* (WARN: exec), gradle_kts_safe/*, gradle_wrapper_safe/*
```

On 6413 files (6412 real) 99.6% pass; every remaining FAIL is a literal by form: eight signing.password=<literal> in Gradle's own docs, a documentation snippet whose comment reads *"you shouldn't"*, two genuine process spawns, and fourteen credential samples caught by the universal rule (credentials { password = "password" } in Gradle's documentation, Spring Boot's "secret", Elasticsearch's dev-run "elastic-password") — two of those before the bracket-expression fix and fourteen after it, none a real secret. The corpus is cloned from gradle/gradle, spring-boot, kafka and elasticsearch, because the reference host has no .gradle file — so it measures what real projects write, not what an AI-generated one does.

Not covered: version catalogs, and -SNAPSHOT versions, which are dependency hygiene rather than an execution vector. gradle-wrapper.jar — a committed binary that ./gradlew runs before any code the repository declares — is handled by the Java archive scanner, §11.2.13.

11.2.9 setup.py and conanfile.py

Matched by name. setup.py runs on a source pip install and a Conan recipe on conan install, so both execute before the package is used. Pure pattern matching; nothing to stage.

They are scanned on their own rules instead of the generic Python ones because the risk differs: in a setup script os.system and subprocess are ordinary build tools, and what is not ordinary is fetching and then executing. The generic rules failed 4 of 36 real setup.py files, three on idioms:

Ordinary — not reported	Reported
`exec(fp.read(), version)` — loading a local version file	`exec(urlopen(url).read())`
`os.system("twine upload dist/*")` — a literal command	`` `os.system("curl … \ ``
`subprocess.check_output(["git", "describe"])`	`exec(payload)` after `b64decode(…)` in the same file
a `build_ext` subclass	a socket handed to `/bin/sh`; a write to `authorized_keys`

Reported as **FAIL**: a download piped into a shell; `exec`/`eval` of a network response; decoding a blob and executing in one file; a socket handed to a shell, or a `pty`; writing `authorized_keys`, a crontab or a systemd unit. Reported as **WARN**: overriding `install` / `develop` / `egg_info`, downloading at install time, and running a program at all.

This replaces the generic Python scan for these two filenames. `bandit` and the generic `eval` / `exec` / `os.system` rules no longer run on them; the attacks they would catch in a setup script are covered by the explicit rules, which measured zero hits on 36 real files. On those files the result moves from 4 FAIL to 0 FAIL, 3 WARN.

`build.rs` is not extended: there is nothing on the reference host to measure against, and `std::process::Command` is how a build script legitimately calls `pkg-config`. It still reaches the Rust scanner, whose `unsafe` and `Command` rules will over-fire on build scripts — a known gap.

```
WORK_DIR=/tmp/setupcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected FAIL: setup_evil/setup.py   Expected PASS (WARN): setup_safe/setup.py
```

11.2.10 Known credential formats — the rule that runs on every file

Providers issue tokens with a fixed shape: a literal prefix, then a body of known length and alphabet. Matching that shape works in **any** file type, quoted or not — `export GITHUB_TOKEN=ghp_…`, a TOML value, a URL, a comment — which the per-type credential rules cannot do, since they expect `key = "value"`. Pure pattern matching; nothing to stage, identical offline.

Provider	Matched on		Provider	Matched on
AWS	`AKIA…`, `aws_secret_access_key`		npm	`npm_…`
GitHub	`ghp_`, `gho_`, `ghu_`, `ghs_`, `ghr_`, `github_pat_`		PyPI	`pypi-AgEIcHlw…`
GitLab	`glpat-`		Hugging Face	`hf_`
Stripe	`sk_live_`, `rk_live_`		Anthropic	`sk-ant-`
Slack	`xoxb-`, `xoxp-`, `xoxa-`, `xoxr-`, `xoxs-`		OpenAI	`sk-proj-`, `legacy sk-`
Google	`AIza…`, `ya29.`		Shopify	`shpat_` and relatives
SendGrid	`SG.…`		DigitalOcean	`dop_v1_`

Each is a **FAIL** (`CWE-798:known_credential_format_<provider>`). A **JWT** is only a **WARN**: it is a signed token rather than necessarily a secret, and Supabase's anon key is one and is public by design.

What it does not do. It is deliberately not a generic `name = value` rule — that fires on 34 of 800 real Python files, because `token = tokens.pop(0)` is an expression. Stripe *test* keys are not matched. Providers whose secrets are bare hex or base64 with no prefix (Heroku, Mailgun, Twilio) are not matched either, since nothing distinguishes them from a hash. Binary files are skipped, so a token inside an image is not seen; jars are the exception (their text resources and class constants are searched, §11.2.13).

False positives. Documentation is full of example keys, so placeholders are excluded: anything containing `EXAMPLE`, `SAMPLE`, `DUMMY`, `YOUR` or `REDACTED`, a run of `x` or `0`, any ten identical characters, and GitHub's documentation token. Each pattern is bounded on both sides, so a token inside a longer identifier does not count. Over 100,052 files of real source, docs and tests plus the host's system directories, the only matches were AWS's own documentation key (18 times, excluded) and three genuine Google API keys hardcoded in `gcloud` and `gsutil`.

Cost: roughly 9% extra scan time, from one additional `grep` per file.

```
WORK_DIR=/tmp/credcheck bash tests/run_tests.sh credential   # builds every token at run time
```

If a vendored file legitimately contains a key (a public Google client key, say), use `.transit-allow.json` with rule `CWE-798` and the file's path.

The name-based credential rule. Alongside the known formats, a rule matches a quoted value after `password`, `secret`, `api_key` and similar names, in every file type, with the same pattern in the YAML, JSON and document branches. It had a bug: inside a bracket expression GNU grep treats `\s` as the literal characters `\` and `s`, so it required four characters none of which was an "s" and missed `password = "sup3rs3cret"`. Fixing it moved the files flagged in 100,052 real source, doc and test files from 66 to 122: 73 newly flagged, almost all test stand-ins such as `"password"` and `"secret"`, and 17 no longer flagged, which were string concatenations such as `"password: " + providedPassword`; no placeholder exclusion was added, because `password = "password"` is exactly the weak default that generated code ships. For test fixtures, use `.transit-allow.json`. Still not matched, and a known limit: upper-case names (`DB_PASSWORD`, `API_KEY`, `SECRET_KEY`) and `secret_key`, because the keyword must be followed directly by `=` or `:` and the match is case-sensitive.

11.2.11 PowerShell — every extension

Pure pattern matching; nothing to stage, identical offline. All PowerShell file types share one rule set:

Extension	What it is	Notes
`.ps1`	script	
`.psm1`	module	
`.psd1`	module manifest	a data hashtable; `ScriptsToProcess` runs code at import
`.pssc`	session configuration	defines a remote session
`.psrc`	role capability (JEA)	a wildcard `VisibleCmdlets` exposes everything
`.ps1xml`	type / format data	XML carrying script blocks; also checked for external entities
`.psc1`	console file	XML; also checked for external entities
`.cdxml`	CIM cmdlet definition	XML; also checked for external entities
(none)	`#!…pwsh` or `#!…powershell` first line	

The legacy Monad extensions (`.msh`, `.msh1`, `.msh2`, `.mshxml`) are not routed; PowerShell has not associated them since 1.0.

Extension case no longer matters. Matching was case-sensitive, so `PAYLOAD.PS1` and `PAYLOAD.PY` passed unscanned while their lower-case twins failed. Extensions are now compared in lower case, for every file type.

What it reports. A construct that is ordinary on its own is a WARN, and becomes a FAIL only in a composition with no innocent reading:

Ordinary — reported, not blocked	Blocks
`Invoke-Expression` on a command it assembles itself (npm's own `npm.ps1`)	`` `irm https://... \ ``
`Start-Process msiexec.exe …` after an https download (Playwright)	`DownloadString` + `iex`, or a shell pipe
`-ExecutionPolicy Bypass`, a hidden window, a scheduled task	`powershell -enc <base64>` (any abbreviation of `-EncodedCommand`)
a `TcpClient` port check	a socket stream fed to `iex` or `cmd.exe`
`$token = $env:GITHUB_TOKEN, $tokenEndpoint = 'https://…'`	`$Password = "literal", ConvertTo-SecureString "…" -AsPlainText`
`FromBase64String` on its own	`FromBase64String` with `iex`, `ScriptBlock::Create`, `Assembly::Load`

It also fails an AMSI bypass, disabling Defender, a `CurrentVersion\Run` key, a WMI event consumer, `VirtualAlloc` with `CreateThread`, offensive tooling by name, and the obfuscated spelling of iex with backticks between its letters. Several of these are whole-file tests, because PowerShell spreads a pipeline across lines.

A deliberate relaxation: `Invoke-Expression` alone is now a WARN. It appeared in three distinct real scripts (npm, npx and pnpm's completion), benign every time. The dangerous compositions above still block; a lone `Invoke-Expression $userInput` does not.

How well-calibrated is it? The previous scanner FAILed 8 of the 69 real scripts on this host (32 distinct) and every one was a false positive; now 0 FAIL and the same eight are WARNs. But 32 distinct scripts is small, and none of the other extensions exists on the reference host, so they are tested from fixtures and not measured. Comments are not stripped, so a payload in a comment is still reported. `.bat` and `.cmd` are covered separately, in §11.2.12.

```
WORK_DIR=/tmp/pscheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected FAIL: ps_download_exec.ps1, ps_irm_iex.ps1, ps_encoded_short.ps1, ps_decode_exec.psm1, …
# Expected WARN (no FAIL): ps_safe_npm_shim.ps1, ps_safe_installer.ps1, ps_manifest.psd1, ps_role.psrc
```

11.2.12 Windows batch — `.bat` and `.cmd`

Pure pattern matching; nothing to stage, identical offline. Before this, every one of the 116 real batch files on the reference host (64 distinct) was recorded PASS, because neither extension had a handler.

Three properties of the language shape the scanner, and all three occur in the real files:

Property	In the real files	What the scanner does
CRLF line endings	104 of 116	strips the CR before matching

Property	In the real files	What the scanner does
`REM` / `::` comment lines	56 of 116	drops them: a commented-out payload does not execute
`^` line continuation	6 of 116	joins the lines, so `certutil ^` / `-urlcache …` is seen as one command

Matching is case-insensitive (`CERTUTIL -URLCACHE`) and so is the extension (`.BAT`).

Ordinary — reported at most as a WARN	Blocks
`certutil -decode cert.b64 cert.cer`	`certutil -decode payload.b64 payload.exe`
`FOR /F … IN ('powershell -command "Get-Date …"')`	`certutil -urlcache -split -f http://…`
`call :setvars` (21 of 116 files)	`curl -o %TEMP%\u.exe https://… && start %TEMP%\u.exe`
`rd /s /q build, del /q /s %BUILDDIR%\*, RD /S /Q C:\build\out`	`rd /s /q C:\`
`set /p PASSWORD= (a prompt), SET "API_KEY=%MY_KEY%"`	`set DB_PASSWORD=hunter2`
`powershell -ExecutionPolicy Bypass -File setup.ps1`	`powershell -enc <base64>`
a licence comment mentioning "a copy of this software"	`c^e^r^t^u^t^i^l …` (carets inside the name)

It also fails a PowerShell download cradle, a `Run` / `RunOnce` / `Winlogon` key, a file copied into the Startup folder, UAC / Defender / the firewall being turned off, `curl -k`, and a new member of the local Administrators group. It warns on `net user /add`, `schtasks /create`, a hidden PowerShell window, `mshta` / `wscript` / `regsvr32`, `wmic process call create`, a remote UNC path, and a command built from adjacent `%v:~n,1%` extractions.

How well-calibrated is it? Less than the 116-file result suggests. The corpus is npm and corepack shims, virtualenv `activate.bat`, Sphinx `make.bat` and Go and Maven wrappers: there is **no Windows-native automation in it**. A clean result there shows the rules do not misfire on benign scripts; it says almost nothing about detecting a hostile one, which rests on fixtures. Across the 116 real files the scanner reports no finding of any kind, and the first version's one false WARN (any two substring extractions on a line, which fired on Maven's `mvn.cmd`) is fixed.

Not handled: batch can build a command from variable substrings at run time, which no pattern can follow; `.btm` (Take Command) and the 4DOS family are not routed.

```
WORK_DIR=/tmp/batcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected FAIL: bat_certutil_dl.bat, bat_continued.bat, bat_UPPER.BAT, bat_ps_cradle.cmd, …
# Expected PASS: bat_safe_real.cmd (WARN: ExecutionPolicy), bat_safe_fetch.bat (WARN: certutil -decode)
```

11.2.13 Java archives — `.jar`, `.war`, `.ear`, `.aar`

Needs only `python3` and its standard library; identical offline. Before this, a `.jar` had no handler, fell through to the unknown-file scanner and was recorded **PASS** whatever it held.

A jar is a ZIP of compiled code. The scanner **never extracts it to disk**: entries are read in place with bounded reads, so a decompression bomb costs a capped read, not the disk. A jar with nothing wrong with it is a **WARN** (`jar:vendored_compiled_code_not_reviewable`) and the verdict stays PASS: bytecode is not decompiled, so what the code *does* is not examined, and a clean result must not read as a review.

Blocks (FAIL)	Reported (WARN)
an entry path with `..`, a leading `/` or `\`, or a drive letter (zip-slip)	native libraries named `.so` / `.dll` / `.dylib`
a symlink entry pointing outside the archive, or whose target cannot be read	a Java agent (`Premain-Class`, `Agent-Class`)
a `.class` entry that does not begin `0xCAFEBABE`	a `.sh` / `.bat` / `.ps1` / `.exe` entry
a PE / ELF / Mach-O stored as `logo.png`	a nested archive, a symlink, duplicate entry names
a native executable glued in front of a valid archive	encrypted or unreadable entries
a local file record the central directory does not list	
a file named `.jar` that is not an archive	a shell launch script before the archive
a manifest `Class-Path` naming an `http(s)://` jar	an absolute `Class-Path`
`` `curl \ ``	`sh`, `/dev/tcp`, `powershell -enc`, `certutil -urlcache` in a resource or class constant
a PEM private key (any line width), or any of the 18 known credential formats	every wrapper jar: verify its checksum
a wrapper jar with extra entries or no `GradleWrapperMain`	

Every limit is reported when reached — an archive that was not fully read must not look like one that was — and each has an environment knob:

Variable	Default	Meaning
`JAR_MAX_ENTRIES`	50000	entries whose bytes are read (entry *names* are always all checked)
`JAR_READ_BUDGET_MB`	64	bytes read across all entries
`JAR_VIEW_MB`	32	text handed to the credential rules
`JAR_MAX_UNCOMPRESSED_MB`	1024	declared size above which a WARN is raised
`JAR_TIME_BUDGET_S`	90	wall-clock limit per archive

Nested archives are opened two levels deep; anything deeper is reported, not opened.

Calibration. 365 real jars (Gradle and Maven distributions, Kotlin, Groovy, native-platform) and the 24 build-tool wrappers (5 distinct) on the reference host. No structural FAIL rule fires on any of them. Reported: 17 bundle native libraries, 2 are agents, 1 nests another archive, 1 exceeds the read budget (`kotlin-compiler-embeddable`, 150 MB uncompressed). One FAILs, correctly: `google-http-client-1.42.2.jar` embeds a real PEM private key. About 0.1 s per jar.

Two real cases shaped it. `0xCAFEBABE` is both the Java class magic and the Mach-O *fat* magic, and `kotlinx-coroutines` ships a class stored as `DebugProbesKt.bin`, so that value is not treated as native. And a PEM check run over one text view, a constant per line, failed `sshd-common-2.12.1.jar` because a 78-character class path looks like a base64 body; it now runs on each string separately.

> **What this does not establish.** The corpus is libraries from two toolchains: it shows the rules do not misfire on real, well-formed jars and says almost nothing about how a hostile one looks, so recall rests on fixtures. `.war`, `.ear` and `.aar` are routed to the same scanner but **none exists on the host** and they are unmeasured. For `gradle-wrapper.jar` the scanner checks *shape*, not contents — the official checksums are not embedded, so a trojaned jar that keeps the `org/gradle/` layout is not detected. That is why every wrapper jar warns: compare its SHA-256 with `https://gradle.org/release-checksums/` before trusting `./gradlew`.

Entries are read the way the JVM and `unzip` read them, not the way a strict library does: a bad CRC, a local-header name that disagrees with the central directory, or a declared size that is too small do not hide an entry, and a local file record the central directory does not list is a FAIL. UTF-16 resources are decoded before the text rules run.

Known gaps. Padding a jar with enough compressible entries to exhaust the read budget yields a WARN (`jar:inspection_incomplete`), not a FAIL — treat that message as "not inspected". A jar under `build/`, `dist/`, `target/`, `vendor/` or `node_modules/` is skipped like every other file there. The shell-pipe and PowerShell rules look 400 and 300 characters back from the pipe / `-enc`. An independent red-team pass over the first version found 15 defects and 3 false positives; the ones that reproduced are fixed, each with a fixture (see CLAUDE.md, "Java archives").

Not covered: bytecode semantics; code in a non-`.class` entry loaded by a custom classloader; JSP web shells inside a `.war`; `.hpi`, `.apk`, `.whl`, `.nupkg` and other ZIP containers.

```
WORK_DIR=/tmp/jarcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected FAIL: jar_zipslip.jar, jar_fake_class.jar, jar_hidden_exe.jar, jar_remote_classpath.jar,
#                jar_hostile_text.jar, wrapper_evil/gradle-wrapper.jar, …
# Expected WARN (no FAIL): jar_safe.jar, jar_agent.jar, jar_native.jar, wrapper_good/gradle-wrapper.jar
```

11.2.14 Office documents — OOXML, OpenDocument, legacy OLE2 and RTF

Needs only `python3` and its standard library; identical offline. Before this, every office document fell through to the unknown-file scanner and was recorded **PASS**.

Routed by extension — `.docx .docm .dotx .dotm .xlsx .xlsm .xlsb .xltx .xltm .xlam .pptx .pptm .ppsx .ppsm .potx .potm .ppam` (OOXML), `.doc .dot .xls .xlt .xla .ppt .pot .pps .ppa` (legacy OLE2), `.odt .ods .odp .odg .ott .ots .otp .otg .fodt .fods .fodp .fodg` (OpenDocument) and `.rtf` — but the **content is sniffed**: Word opens an RTF named `.docx` without complaint. A file that is not an office document at all goes back to the unknown-file path when it is plain text and plausibly something else (`.dot` is Graphviz, `.pot` is gettext, a `.doc` can be text) and is reported otherwise.

The document is **never extracted to disk**; a standard-library helper reads it in place. An office document is data, so one with nothing to report is a **PASS** (a jar, which is code, is a WARN).

The XML is read the way a parser would hand it over, not the way it is spelled: element and attribute names are compared **without their prefix** (a prefix is arbitrary, so `x:instrText` is a field instruction as `w:instrText` is), the encoding comes from the byte-order mark or declaration (UTF-16 parts are decoded), comments and CDATA are normalised away, and a part is recognised by what `[Content_Types].xml` and the relationships say it is, so a main part, macro sheet or VBA project works wherever it is stored. Flat OPC, Word 2003 XML and Excel 2003 XML are read whatever the extension. Word opens an RTF whatever the file is called, so a `.txt`, `.log`, `.md`, `.rst` or extensionless file whose first kilobyte contains `{\rtf` is scanned as an RTF, and a `.xml` that is Flat OPC or Word/Excel 2003 XML is scanned as a document as well as an XML file.

Blocks (FAIL)	Reported (WARN)
a VBA macro that starts a program or downloads (the module is decompressed and read)	any VBA macro, one that runs on open, one in a `.docx`/`.xlsx`/`.pptx`
a DDE / `DDEAUTO` field, or a DDE formula, `ddeLink`, ODF `DDE()` call or legacy `.xls` SUPBOOK link whose service **or topic** names a shell or a program (`=MSEXCEL\	'..\cmd.exe /c ...'!A0`)
an Excel 4.0 macro sheet that is hidden or calls `EXEC`/`CALL`/`REGISTER`	a visible Excel 4.0 macro sheet
a remote `attachedTemplate`, or an `oleObject`/`frame`/`package` fetched from a remote host	content fetched from http(s) on open; a hyperlink to a file share
an `ms-msdt:`, `mhtml:`, `search-ms:`, `javascript:` or `vbscript:` link	ActiveX controls, embedded OLE objects that cannot be read as an office document
a UNC path (`\\host\share`, `smb://`) outside a hyperlink (NTLM leak); an `INCLUDETEXT`/`INCLUDEPICTURE`/`LINK` field aimed at a remote host or UNC path; a `HYPERLINK` field with an `ms-msdt:`/`javascript:` target	an Equation object without a payload
an embedded executable or script, also under a document name	an alternate-format chunk; a bare `DOCTYPE`
an Equation Editor object carrying a command or URL; a PowerPoint `ppaction://program` hyperlink; a legacy `.doc` object holding a URL moniker to a remote address	an ODF macro, or a hyperlink that runs one; a `ppaction://macro` action; a hyperlink to an executable
an external `ENTITY`; a part named `../…`	an ODF resource linked from http(s) or a path
RTF: a linked object (flags, `OLE2Link` class, or URL moniker) whose target URL — ASCII or UTF-16 — is remote (CVE-2017-0199); a remote `\template`	an RTF embedded OLE object
ODF: a document-embedded macro bound to `office:load` / `office:new`; a script calling `Shell`, `os.system` …	encrypted content, an unparseable container, an unreadable VBA project
a PEM private key or any of the 18 known credential formats in the text	every limit reached

Prose is not searched for hostile strings: a manual that says "pipe curl into sh" is ordinary. The `curl | sh` / encoded-PowerShell / `certutil` rules run on macro source, field codes, formulas and ODF scripts only.

Every limit is reported when reached, and each has an environment knob. The time budget is checked between parts; a pattern run away inside one part is stopped by an **alarm** armed at the budget plus ten seconds in both this helper and the jar helper, which prints what was found and reports the document as `inspection_incomplete`. Reads from a compound file are charged to the read budget too.

Variable	Default	Meaning
`OFFICE_MAX_ENTRIES`	20000	parts whose bytes are read (part *names* are always all checked)
`OFFICE_READ_BUDGET_MB`	256	bytes read across all parts
`OFFICE_TIME_BUDGET_S`	90	wall-clock limit per document

Objects embedded in a document are opened three levels deep; anything deeper is reported, not opened. An embedded object that is itself an office document (the workbook behind every chart, an

OOXML package wrapped in an OLE `Package` stream, the OLE objects of a legacy `.ppt`) is analysed like the top-level file: clean is unremarkable and a macro inside it is a FAIL. Only an object that stays opaque is reported. pandoc writes RTF *fragments* with no `{\rtf` header; those are read, and warned about, as plain text.

Calibration. 123 real OpenDocument templates, the python-docx and python-pptx defaults, an RTF, and 62 documents LibreOffice converted to every format: **187 documents, 17 extensions, no finding**; a second set of 30 with charts, pictures and tables found (and, once fixed, passes clean) the embedded-workbook noise. The real files changed the design: every OpenDocument file declares the `script:` namespace (a rule on it would fail 100%), **10 of the 123 templates bind an event to a macro the application ships** (`application:…`, not a finding), and `meta:template` carries a `file:///` link in every document made from a template.

> **What this does not establish.** None of the 217 has a macro, a DDE field or a remote link, so what the rules do on a hostile document rests on fixtures. They are validated against `olevba`, `olefile` and `rtfobj` (independent readers), not just against this scanner. Obfuscated VBA, VBA stomping and password-protected files are not read (the last is reported). A document padded past the read budget is a WARN (`office:inspection_incomplete`), not a FAIL: treat that message as "not inspected".

Not routed: `.one`, `.pub`, `.vsd`, `.msg`, `.csv`/`.tsv` formula injection, and anything that merely contains a document (`.zip`, `.iso`, an `.eml` attachment).

```
WORK_DIR=/tmp/officecheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected FAIL: office_macro_dangerous.docm, office_remote_template.docx, office_dde.docx, office_rtf_equation.rtf, …
# Expected PASS: office_safe.docx, office_safe.odt (an application: event listener), office_graphviz.dot
```

11.3 Environment variables

Variable	Default	Purpose
`OFFLINE`	`false`	Same as `--offline`
`OFFLINE_CACHE`	`$WORK_DIR/offline-cache`	Root of the staged assets
`SEMGREP_RULES_DIR`	`$OFFLINE_CACHE/semgrep-rules`	Exported ruleset YAML files
`TRIVY_CACHE_DIR`	`$OFFLINE_CACHE/trivy-db`	trivy database cache
`CLAMAV_DB_DIR`	`$OFFLINE_CACHE/clamav`	ClamAV signature files

`--offline` also exports `SEMGREP_SEND_METRICS=off`, `DO_NOT_TRACK=1`, `CHECKPOINT_DISABLE=1` and `PIP_NO_INDEX=1`, so no tool blocks on a telemetry or version-check call.

11.4 Building the cache — on a connected host

```
./prepare_offline_cache.sh /path/to/offline-cache
```

It stages the Semgrep rulesets, the trivy database, the ClamAV signatures and your YARA rules, writes a SHA-256 manifest, and prints a summary of what it could and could not stage.

Expect roughly **300–800 MB**, dominated by the trivy database.

11.5 Transferring and verifying

```
# connected host
tar -czf offline-cache.tar.gz -C /path/to offline-cache
sha256sum offline-cache.tar.gz          # note this value
```

```
# air-gapped host, after transfer
sha256sum offline-cache.tar.gz          # must match
tar -xzf offline-cache.tar.gz -C /opt/ai-transit/
cd /opt/ai-transit/offline-cache && sha256sum --check .cache_manifest.sha256
```

11.6 Running

```
export OFFLINE_CACHE=/opt/ai-transit/offline-cache
./ai_transit.sh --offline /path/to/repo

# Docker: the wrapper mounts OFFLINE_CACHE read-only at /offline-cache in the container.
# The image stages no Semgrep rules, so layer 2 only runs when the cache is mounted.
OFFLINE_CACHE=/opt/ai-transit/offline-cache \
  ./docker-run.sh --offline /path/to/repo
```

11.7 Confirming a clean offline run

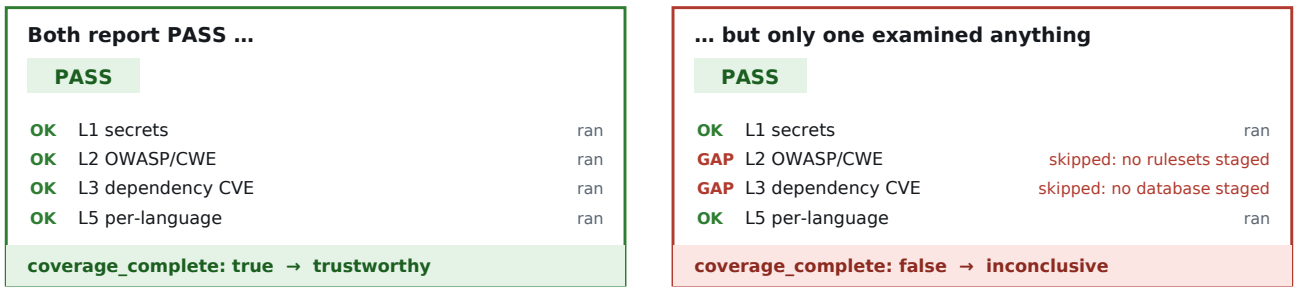
The report should contain **no** `OFFLINE:` warning about Layer 2 or trivy:

```
grep -o 'OFFLINE:[^|]*' "$WORK_DIR"/reports/report_*.json
```

Warning seen	Meaning	Fix
`Layer 2 skipped entirely`	No rulesets staged — OWASP/CWE did not run	Re-run `prepare_offline_cache.sh`
`semgrep rulesets not staged (...)`	Some rulesets missing — partial coverage	Export the named rulesets
`trivy database not staged`	No dependency CVE coverage at all	Stage the trivy DB
`ClamAV signature database is empty`	Malware scan found nothing because it had no signatures	Stage `*.cvd` files
`Python/JavaScript dependency CVE scan unavailable`	Expected — see Group C	None; trivy covers this

The `trivy database not staged` and dependency-CVE warnings only appear when layer 3 is enabled (`RUN_SCA=true`); by default it is recorded as `skipped:disabled` instead.

11.8 Coverage — proving the scan actually ran



A verdict alone cannot distinguish "clean" from "nothing was examined".
Automated consumers should gate on the coverage block, not on the verdict.

Figure 6 — two reports that both say PASS; only one is trustworthy.

Every JSON report carries a `coverage` block stating, per layer, whether it ran. This exists because a verdict alone cannot distinguish "clean" from "nothing was examined", and offline that distinction is

easy to lose.

```
"coverage": {
  "L1_secrets_gitleaks": "ran",
  "L1_malware":              "skipped:no ClamAV signatures available",
  "L2_owasp_cwe":            "ran",
  "L3_dependency_cve":       "skipped:trivy database not staged for offline use",
  "L4_patterns":             "ran",
  "L5_per_language_sast":    "ran",
  "L6_licence":              "ran"
},
"coverage_complete": false,
"coverage_gaps":     ["L1_malware", "L3_dependency_cve"]
```

A layer can fail to run for two unrelated reasons — the tool is not installed, or its data was not staged — and the block records both the same way, so a consumer does not have to parse warning text to tell them apart.

Automated consumers should gate on coverage, not on the verdict:

```
REPORT=$(ls -t "$WORK_DIR"/reports/report_*.json | head -1)
python3 -c "
import json,sys
d=json.load(open(sys.argv[1]))
print('verdict :', d['verdict'])
print('gaps    :', ', '.join(d['coverage_gaps']) or 'none')
sys.exit(0 if d['coverage_complete'] else 2)
" "$REPORT"
```

`OFFLINE_RUNBOOK.md` contains a ready-made acceptance gate keyed on the five layers a verdict most depends on.

11.9 Keeping the cache current

Vulnerability data ages. A stale trivy database reports a clean result for CVEs published after it was built, which is the same failure mode as not scanning at all — only harder to notice.

Rebuild and re-transfer the cache on a defined cadence (**weekly** is a reasonable default for the trivy DB and ClamAV signatures; Semgrep rulesets change more slowly). Record the build date alongside the archive so the air-gapped side can tell how old its data is.

12. Installing on a Disconnected Host

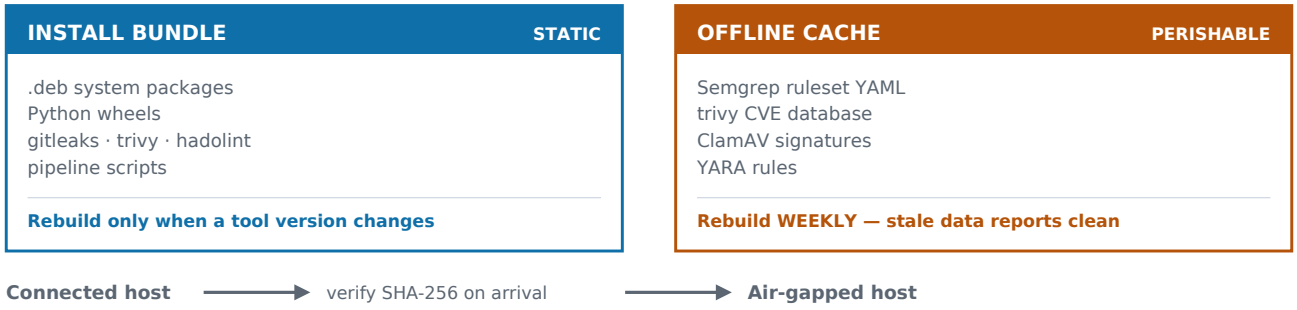


Figure 5 — the two transfer bundles and their differing lifetimes.

Section 10 covers running scans without a network. This section covers the step before it: getting the pipeline and its tools onto a machine that has never had one.

The distinction matters operationally, because the two bundles have different lifetimes:

Bundle	Contains	Rebuild when
Install bundle (this section)	The software: tools, binaries, pipeline scripts	A tool version changes
Scan cache (§11.4)	The data scanners read: rules, CVE database, signatures	**Weekly** — it is perishable

Every command in sections 3–6 assumes a network. `apt-get install` reaches archive.ubuntu.com, `pip install` reaches pypi.org, `go install` reaches the Go module proxy, and `docker build` reaches all of them plus Docker Hub. None of that works on an isolated host.

12.1 Choose a path

Path	Effort	Best when
A — Docker image transfer	Low	Docker is permitted on the target. Strongly preferred.
B — Native package staging	High	Docker is not permitted, or the target must run the tools directly.

Path A moves one file and is far less error-prone: the image already contains all sixteen tools at known versions, so nothing can be partially installed.

> **Both paths require a connected build host running the same OS release and CPU architecture as the target.** Debian packages and many Python wheels are compiled artefacts — an amd64 Ubuntu 22.04 bundle will not install on an arm64 host or on Debian 12, and the failures are confusing rather than obvious.

12.2 Path A — Docker image transfer

On the connected host:

```
./docker-run.sh --build                      # build once
./prepare_offline_install.sh /tmp/offline-install
```

That exports the image with `docker save` into the bundle. To do it by hand:

```
docker save ai-transit:latest | gzip > ai-transit.tar.gz
sha256sum ai-transit.tar.gz | tee ai-transit.tar.gz.sha256
```

Expect roughly **2–3 GB** compressed.

On the air-gapped host:

```
sha256sum -c ai-transit.tar.gz.sha256        # must pass before loading
gunzip -c ai-transit.tar.gz | docker load
docker image ls ai-transit                   # confirm it is present

# Verify it runs and is non-root
docker run --rm --entrypoint id ai-transit:latest -u    # must not be 0
```

The pipeline scripts are inside the image, but you still need them on the host to use `docker-run.sh` and to hold the scan cache. Copy `pipeline/` from the bundle to your install directory.

12.3 Path B — Native package staging

On the connected host (same OS and architecture as the target):

```
./prepare_offline_install.sh /tmp/offline-install
```

It stages four groups:

Directory	Contents	How it was obtained
`deb/`	System packages and their dependencies	`apt-get install --download-only`
`wheels/`	Python packages	`pip3 download`
`bin/`	gitleaks, trivy, hadolint	Copied from the build host's PATH
`pipeline/`	Scripts, tests, documentation	Copied from the repository

`bin/` is populated from the tools already installed on the build host, so install them there first (§6) before running the script.

Transfer and verify:

```
tar -czf offline-install.tar.gz -C /tmp offline-install
sha256sum offline-install.tar.gz | tee offline-install.tar.gz.sha256

# On the air-gapped host
sha256sum -c offline-install.tar.gz.sha256
tar -xzf offline-install.tar.gz -C /opt/
cd /opt/offline-install
sha256sum --check .install_manifest.sha256 | grep -v ': OK$' || echo "all files OK"
```

Check the platform matches before installing anything:

```
cat .bundle_platform
# os_id=ubuntu  os_version=22.04  arch=amd64  built_on=...

. /etc/os-release; echo "target: ${ID} ${VERSION_ID} $(dpkg --print-architecture)"
# These must agree. If they do not, rebuild the bundle on a matching host.
```

Install:

```
# 1. System packages — dpkg resolves nothing, so install the whole set at once
sudo dpkg -i deb/*.deb || sudo apt-get install -f --no-download -y

# 2. Python packages — --no-index forbids any network fallback
python3 -m venv /opt/ai-transit/venv
source /opt/ai-transit/venv/bin/activate
pip install --no-index --find-links=wheels \
    openpyxl reportlab python-docx detect-secrets bandit \
    pip-audit safety semgrep checkov scancode-toolkit

# 3. Binaries
sudo cp bin/* /usr/local/bin/ && sudo chmod +x /usr/local/bin/{gitleaks,trivy,hadolint}

# 4. Pipeline
```

```
sudo mkdir -p /opt/ai-transit
sudo cp -r pipeline/* /opt/ai-transit/
sudo chmod +x /opt/ai-transit/*.sh
```

`pip install --no-index` is deliberate: without it, pip silently falls back to PyPI and the install appears to succeed on a host that merely has *partial* network access, hiding the fact that the bundle was incomplete.

12.4 Verify the installation

```
cd /opt/ai-transit

# 1. Which tools actually made it
for t in gitleaks detect-secrets clamscan yara semgrep trivy \
         bandit shellcheck cppcheck hadolint checkov scancode; do
    command -v "$t" >/dev/null && echo "  present : $t" || echo "  MISSING : $t"
done

# 2. The suite needs no tools and no network — it should pass regardless
./tests/run_tests.sh
# Expected: ✔ 740/740 passed

# 3. Record the bundle for the integrity check
python3 selfcheck.py --write-manifest
```

Anything reported `MISSING` will WARN on every scan and never contribute findings. Resolve it now rather than discovering it in a report later — §11.8 explains how the coverage block makes such gaps visible.

12.5 Then stage the scan data

Installing the tools is only half the job. The scanners still need their rules and databases, which is a **separate and perishable** bundle:

```
# On the connected host
./prepare_offline_cache.sh /tmp/offline-cache
```

Follow §11.4–10.6, or the step-by-step procedure in **OFFLINE_RUNBOOK.md**.

13. Sample Scans — Testing the Pipeline

13.1 Quick smoke test (local directory)

Create a small test repository to validate the pipeline end-to-end:

```
#!/usr/bin/env bash
# create_test_repo.sh — creates a clean sample repo that should PASS

mkdir -p /tmp/sample-repo/{src,tests,infra}

# Python source
cat > /tmp/sample-repo/src/app.py << 'EOF'
import os
import sqlite3
```

```
def get_user(db_path: str, username: str) -> list:
    """Retrieve user using parameterised query (safe)."""
    conn = sqlite3.connect(db_path)
    return conn.execute(
        "SELECT * FROM users WHERE name = ?", (username,)
    ).fetchall()

def main():
    db = os.environ.get("DB_PATH", "/tmp/app.db")
    print(get_user(db, "alice"))
EOF

# Shell script
cat > /tmp/sample-repo/src/setup.sh << 'EOF'
#!/usr/bin/env bash
set -euo pipefail
APP_DIR="${APP_DIR:-/opt/app}"
mkdir -p "$APP_DIR"
echo "Setup complete"
EOF

# Requirements file (up-to-date)
cat > /tmp/sample-repo/requirements.txt << 'EOF'
requests==2.32.3
Pillow==10.3.0
EOF

# README
echo "# Sample project" > /tmp/sample-repo/README.md

echo "Test repo created at /tmp/sample-repo"
```

```
bash create_test_repo.sh
./ai_transit.sh /tmp/sample-repo
# Expected: PASS — ZIP produced in ./Good/
```

13.2 FAIL test — embedded secret

```
mkdir -p /tmp/fail-secret

cat > /tmp/fail-secret/config.py << 'EOF'
# Bad practice: hardcoded credential
EOF
# The key is assembled at run time, so this document never holds a contiguous token.
# (AWS's documentation key, AKIAIOSFODNN7EXAMPLE, is allowlisted by every scanner.)
printf 'AWS_ACCESS_KEY_ID = "AKIA%s"\n' "QYLPMN5HHHFPZAM2" >> /tmp/fail-secret/config.py

./ai_transit.sh /tmp/fail-secret
# Expected: FAIL — gitleaks and the known-credential-format rule (CWE-798) both detect the AWS key
# Files moved to /opt/ai-transit/quarantine/

rm -rf /tmp/fail-secret
```

13.3 FAIL test — SQL injection (OWASP A03)

```
mkdir -p /tmp/fail-sqli

cat > /tmp/fail-sqli/db.py << 'EOF'
import sqlite3

def login(username, password):
    conn = sqlite3.connect("app.db")
    query = "SELECT * FROM users WHERE name='" + username + \
            "' AND pass='" + password + "'"
    return conn.execute(query).fetchall()
EOF

./ai_transit.sh /tmp/fail-sqli
# Expected: FAIL — Semgrep detects SQL injection (OWASP A03 / CWE-89)

rm -rf /tmp/fail-sqli
```

13.4 FAIL test — vulnerable dependency (CVE)

```
mkdir -p /tmp/fail-cve

cat > /tmp/fail-cve/requirements.txt << 'EOF'
Pillow==9.0.0
requests==2.18.0
EOF

./ai_transit.sh /tmp/fail-cve
# Expected: FAIL — trivy/pip-audit detect CVEs in Pillow 9.0.0

rm -rf /tmp/fail-cve
```

13.5 WARN test — risky licence (GPL)

```
mkdir -p /tmp/warn-licence

cat > /tmp/warn-licence/main.py << 'EOF'
# This file is licensed under the GNU General Public License v3.0
# Copyright (C) 2024 Example Corp.
# You may redistribute and/or modify under the terms of the GPL.

def hello():
    print("hello")
EOF

./ai_transit.sh /tmp/warn-licence
# Expected: PASS (WARN logged) — ScanCode detects GPL-3.0 licence
# ZIP produced in ./Good/ but WARN logged for legal review

rm -rf /tmp/warn-licence
```

13.6 FAIL test — Dockerfile misconfigurations

```
mkdir -p /tmp/fail-docker

cat > /tmp/fail-docker/Dockerfile << 'EOF'
FROM ubuntu:latest
ENV DB_PASSWORD=mysecretpassword
RUN apt-get update && apt-get install -y curl
ADD https://example.com/app.tar.gz /app/
EOF

./ai_transit.sh /tmp/fail-docker
# Expected: FAIL — hadolint detects :latest tag, ENV secret, ADD from URL

rm -rf /tmp/fail-docker
```

13.7 Scan a real public GitHub repository

```
# Scan a small, well-known public repo (adjust URL as needed)
./ai_transit.sh https://github.com/psf/requests main
# Expected: PASS or WARN — clean repo, dependencies may have minor CVEs
```

13.8 Run the self-check to verify the pipeline itself

```
# PDF report (default)
python3 selfcheck.py --bundle-dir . --output selfcheck_report.pdf

# JSON report (machine-readable, CI-friendly)
python3 selfcheck.py --bundle-dir . --format json --output selfcheck_report

# Both formats at once
python3 selfcheck.py --bundle-dir . --format both --output selfcheck_report

# Run only specific checks (faster)
python3 selfcheck.py --bundle-dir . --only 11.1,11.4,11.6

# Expected output:
#   11.1 Meta-scan        → PASS or WARN
#   11.2 Binary checks    → SKIP (unless --checksums provided)
#   11.3 GPG/cosign       → PASS or WARN
#   11.4 Python CVE       → PASS
#   11.5 Host OS CVE      → PASS or WARN
#   11.6 Bundle integrity → PASS
#   11.7 AIDE             → SKIP (unless AIDE installed)
```

13.9 CI mode — quiet verdict with severity filter

```
# Only block on CRITICAL CVEs/findings; warnings and lower are ignored
./ai_transit.sh --quiet --min-severity critical https://github.com/org/repo
echo "Exit code: $?"   # 0 = PASS, 1 = FAIL
```

13.10 Diff mode — scan only changed files (PR workflow)

```
# Scan only files changed since the merge base commit
SINCE=$(git merge-base HEAD origin/main)
```

```
./ai_transit.sh --since "$SINCE" /path/to/local/repo
```

13.11 Private repository scan

```
# Generate a fine-grained PAT with "Contents: read" on github.com
export GITHUB_TOKEN="ghp_your_token_here"
./ai_transit.sh https://github.com/myorg/private-repo main
```

13.12 Report-only mode (audit without blocking)

```
# Scan and generate reports but always exit 0 — useful for first-pass auditing
./ai_transit.sh --report-only https://github.com/org/repo
# Reports are written to $WORK_DIR/reports/ regardless of findings
```

13.13 Air-gapped scan

```
# On a connected host: build the cache and transfer it
./prepare_offline_cache.sh /tmp/offline-cache
tar -czf offline-cache.tar.gz -C /tmp offline-cache

# On the air-gapped host
tar -xzf offline-cache.tar.gz -C /opt/ai-transit/
cd /opt/ai-transit/offline-cache && sha256sum --check .cache_manifest.sha256

export OFFLINE_CACHE=/opt/ai-transit/offline-cache
./ai_transit.sh --offline /path/to/repo

# Confirm no layer was silently skipped
grep -o 'OFFLINE:[^|]*' "$WORK_DIR"/reports/report_*.json
# Expected: only the Python/JavaScript dependency notices (see 10.2 group C)
```

14. Self-Scan: Verifying the Installation is Safe

Before deploying in a production environment, run the full self-check:

```
python3 selfcheck.py --bundle-dir . --output selfcheck_report.pdf
```

This executes all self-check checks (meta-scan, binary checksums, GPG/cosign, Python CVE scan, host OS CVE, bundle integrity, AIDE) and produces a colour-coded PDF report. See `selfcheck.py --help` for all options.

14.1 Output formats and selective checks

```
python3 selfcheck.py --format json --output selfcheck_report   # machine-readable
python3 selfcheck.py --format both --output selfcheck_report   # PDF + JSON
python3 selfcheck.py --only 11.1,11.4,11.6                     # subset, faster
```

The JSON report carries a top-level `verdict` field (`PASS` / `WARN` / `FAIL`), which is what a CI job should key on.

14.2 The bundle manifest — generate it at install time

Self-check 11.6 (bundle file integrity) compares every bundle file against `.bundle_manifest.sha256`. That manifest is **deliberately not tracked in version control**: if it were, it would report "File tampering detected" after every ordinary edit, and a check that cries wolf is a check people learn to ignore. Git already guarantees the integrity of the repository; the manifest's job is to detect tampering with an **installed** bundle.

Generate it once, immediately after installing and verifying the bundle:

```
python3 selfcheck.py --write-manifest
```

Regenerate it after any *intentional* change to the bundle. From then on, any self-check 11.6 failure means a file changed without your knowledge.

```
# Verify (should PASS on an untouched installation)
python3 selfcheck.py --only 11.6
```

The manifest stores **relative** paths, so it stays valid if the bundle is moved or installed to a different prefix. Verification must therefore be run from the bundle directory (selfcheck.py does this automatically).

15. Security Hardening Recommendations

Dedicated service account

```
sudo useradd -r -m -d /opt/ai-transit -s /bin/bash aitransit
sudo chown -R aitransit:aitransit /opt/ai-transit
sudo -u aitransit ./ai_transit.sh https://github.com/org/repo
```

Network isolation (allow only github.com outbound)

```
sudo ufw default deny outgoing
sudo ufw allow out 443 comment "HTTPS for GitHub"
sudo ufw allow out 53  comment "DNS"
sudo ufw enable
```

Quarantine filesystem (noexec)

```
# /etc/fstab
tmpfs /opt/ai-transit/quarantine tmpfs rw,noexec,nosuid,nodev,size=2G 0 0
```

Weekly tool updates (cron)

```
# /etc/cron.weekly/ai-transit-update
#!/bin/bash
freshclam
trivy image --download-db-only
source /opt/ai-transit/venv/bin/activate
pip install --upgrade detect-secrets semgrep bandit \
    pip-audit safety checkov scancode-toolkit
```

Log rotation

```
# /etc/logrotate.d/ai-transit
/opt/ai-transit/logs/*.log {
    daily
    rotate 30
    compress
    missingok
    notifempty
}
```

16. Troubleshooting

Symptom	Likely cause	Fix
`gitleaks: command not found`	Binary not in PATH	`sudo cp ~/go/bin/gitleaks /usr/local/bin/`
`freshclam: connect refused`	clamav-freshclam not running	`sudo systemctl start clamav-freshclam`
`semgrep: network error`	No internet — first run	Pre-download rules (see §11)
`trivy: DB not found`	First run without internet	`trivy image --download-db-only`
`scancode: takes too long`	Large repo	Use `--timeout 30` to limit per-file time
`pip-audit: no vulnerabilities` on old packages	DB unreachable	Check internet; use trivy offline instead
`npm audit: ENOLOCK`	No package-lock.json	Run `npm install --package-lock-only` first
`openpyxl manquant`	Not installed in venv	`pip install openpyxl`
`declare -A: invalid option`	Bash < 4.0	Install bash 5: `brew install bash` / use WSL2
`zip: command not found`	zip not installed	`sudo apt-get install zip`
WARN on all files	Missing optional tools	Install missing tools; WARNs do not block
Git clone fails on a private repo	No credentials	Set `GITHUB_TOKEN` (see §9.2); the token is passed via `GIT_ASKPASS`, never in the clone URL
`Repository not found or private (HTTP 404)`	Private repo without a token, or a typo	Set `GITHUB_TOKEN`, or check the URL
self-check 11.6 reports "File tampering detected" after your own edits	Manifest is stale	Regenerate it: `python3 selfcheck.py --write-manifest` (see §13.2)
`--only` reports 0 checks and exits 2	Check IDs mistyped (`1.1` instead of `11.1`)	Use the full IDs: `11.1` … `11.7`
Findings appear against the wrong file	Report written by a pre-P8 version	Upgrade; the JSON writer dropped empty records and shifted rows
Allowlist entries have no effect	`.transit-allow.json` not at the repository root, or `rule`/`path` do not match	`path` is relative to the repo root; `rule` matches the finding's leading token, e.g. `CWE-89`

Symptom	Likely cause	Fix
Diff mode scans everything	`--since` commit unreachable in a shallow clone	The pipeline warns and falls back to a full scan; fetch more history or use a local path
Docker build fails downloading trivy	Pinned version no longer published	Update `ARG TRIVY_VERSION` / `TRIVY_SHA256` (see §19.1); the `pins` CI job prints the correct digest
ZIP contains `tmp/…/fetch/repo_…` paths	Archive built by a pre-P8 version	Upgrade; archives are now rooted at the repository
Fetching hangs for minutes on an isolated host	The fetch step (not the scan) is waiting for the network	Use `--offline` with a local path (see §11); the scan itself is always offline
Offline run passes suspiciously fast with few findings	Layers 2 and 3 had no staged data	Check for `OFFLINE:` warnings in the report; stage the cache (§11.4)
`OFFLINE:Layer 2 skipped entirely`	Semgrep rulesets not staged	Run `prepare_offline_cache.sh` on a connected host
`OFFLINE:trivy database not staged`	No dependency CVE coverage offline	Stage the trivy DB; it is the only offline CVE source
`Cannot clone … without a network`	`--offline` with a remote URL	Copy the repository to the host and pass its path

17. Running the Test Suite

The suite verifies the pipeline itself: that rules fire on unsafe code, that they **do not** fire on safe code, that flags behave, and that the reports and archive are well-formed.

```
./tests/run_tests.sh          # everything
./tests/run_tests.sh -v       # show detail for failures
./tests/run_tests.sh rules    # only groups whose name matches "rules"
```

It requires **no scanning tools at all**. With none installed the pipeline degrades to its built-in grep rules and every assertion still holds — that is exactly how CI runs it. Tools that happen to be present are used, but no assertion depends on them.

Expected output on a healthy installation:

```
── Layer A — rule corpus (detection correctness)
  ✔ sql_injection.py is flagged for SQL injection
  ✔ does NOT flag parameterised SQL (execute("… = ?", (v,)))
  …
──────────────────────────────────────────────
  ✔  740/740 passed
```

17.1 What each layer covers

Layer	Covers
A — rule corpus	Detection correctness: each finding must land on the correct file, plus false-positive guards for safe code

Layer	Covers
B — end-to-end	Clean repo → PASS/exit 0; vulnerable repo → FAIL/exit 1
C — flags	`--report-only`, `--min-severity`, argument guards, `.transit-allow.json`, `.transitignore`, `--no-zip`/`--no-excel`
D — artifacts	JSON valid and carries a verdict, HTML written and escaped (a hostile file or jar-entry name is shown as text), ZIP entries repo-relative, Excel Findings tab, no escape codes when redirected, no terminal control bytes taken from a file name
Java archive limits	The `JAR_*` environment limits of `scan_jar`, each reached and reported on an 11 KB fixture, plus the archive helper's behaviour on a file that cannot be opened
Office document limits	The `OFFICE_*` limits of `scan_office`, six adversarial inputs that must finish inside 20 s (quadratic patterns), and a DDE field whose instruction lies in a later 4 MB slice than its `begin`
E — diff mode	`--since` scans exactly the changed files; `.git` excluded from local copies
G — customised deployment	Optional `report_dir` / `scanning_name`, no online mode, gitleaks run on the pipeline's terms (a repository's own `.gitleaks.toml`, `.gitleaksignore` and inline allow comments are ignored), structured findings and the new report fields, layers 3 and 6 off by default and recorded as `skipped:disabled`, exit status 1 unless PASS, hostile inputs (non-UTF-8 names, stale `.diff_files`, empty allowlist, trailing slash, unusable names)
F — static	Parse checks, shellcheck, and lint rules for the bug classes that have already shipped (six silent-failure classes, `echo -e` on repository text, and an associative array measured with `${#…}` without being initialised)

17.2 Fixtures

`tests/fixtures/` holds small repositories, each with one job:

Fixture	Expected
`clean/`	PASS
`vulnerable/`	FAIL
`allowlisted/`	PASS — the finding is downgraded by `.transit-allow.json`
`ignored/`	PASS — the offending file is excluded by `.transitignore`
`rules/`	A corpus where each file triggers, or deliberately does not trigger, one rule

`rules/safe_sql.py` is a regression guard: it contains correct parameterised queries that a previous version of the SQL rule wrongly flagged as injection.

17.3 Adding a rule

Add **both** a file that must trigger the rule and a similar-but-safe file that must not, then confirm the new assertion **fails before the rule exists**. A test that has never been observed failing proves nothing — during development of this suite, two tests passed against known-broken code because the tests themselves were wrong.

18. Continuous Integration

`.github/workflows/ci.yml` runs on every push and pull request.

Job	Purpose	Blocking
`lint`	shellcheck (errors fatal, warnings advisory) + Python syntax	Yes
`test`	Test suite with no scanning tools — the fast signal	Yes
`test-with-tools`	Suite again with semgrep/bandit/detect-secrets installed; exercises tool-dependent paths the degraded run cannot reach	No
`pins`	Downloads each pinned tool artifact, fails if the version does not exist, and compares against the pinned SHA-256	Yes
`docker`	Builds the image, asserts it runs as non-root, smoke-tests both fixtures	Yes

The `docker` job is the only place the multi-stage build, the pinned tool versions and the non-root user are actually exercised — none of that can be verified by the test suite alone.

If `pins` fails, the pinned version in the `Dockerfile` no longer resolves to a downloadable artifact. The job log prints the correct SHA-256 for the current pin; update `ARG TRIVY_VERSION` / `ARG TRIVY_SHA256` accordingly.

19. Docker Image — Build Arguments & Integrity

The image is a two-stage build: a `builder` stage compiles gitleaks with Go, and the runtime stage copies only the resulting binary, so the Go toolchain never reaches the final image. It runs as the non-root user `transit`.

19.1 Build arguments

Argument	Default	Purpose
`TRIVY_VERSION`	`0.58.2`	trivy release to install
`TRIVY_SHA256`	*(empty)*	SHA-256 of the trivy tarball; empty disables verification
`HADOLINT_VERSION`	`2.12.0`	hadolint release to install
`HADOLINT_SHA256`	`56de6d5e…`	SHA-256 of the hadolint binary (verified)
`GITLEAKS_VERSION`	`8.30.1`	gitleaks module version

```
docker build -t ai-transit:latest \
  --build-arg TRIVY_VERSION=0.58.2 \
  --build-arg TRIVY_SHA256=<digest> .
```

19.2 Why the digests matter

Pinning a version defends against getting a *different release*. It does not defend against getting a *different binary* for that release — a compromised or replaced artifact keeps the same version string. The digest closes that gap.

When a digest argument is empty the build still succeeds, but prints a warning **and the artifact's actual digest**, so the correct value can be pasted in without a separate download. The `pins` CI job prints the same value.

> `TRIVY_SHA256` ships empty because the pinned trivy version could not be resolved from the development environment. Fill it in from the first `pins` CI run before treating the image as production-ready.

19.3 Running through the wrapper

`docker-run.sh` forwards pipeline flags into the container, forwards `RUN_SCA` and `RUN_SCANCODE`, and mounts `OFFLINE_CACHE` read-only at `/offline-cache`. The image itself stages no Semgrep rulesets, so without that mount layer 2 does not run (the report says so). With the cache mounted the Docker and native interfaces behave alike:

```
./docker-run.sh --build                                          # build once
./docker-run.sh --quiet --min-severity critical https://github.com/org/repo
./docker-run.sh --report-only /path/to/local/repo
```

Environment variables (`WORK_DIR`, `MAX_SIZE_MB`, `MIN_SEVERITY`, `VERBOSITY`, `GITHUB_TOKEN`, `SINCE_COMMIT`) are forwarded automatically when set.

> A `GITHUB_TOKEN` passed to a container is visible via `docker inspect` to anyone who can reach the Docker daemon. On a shared host, prefer running the pipeline natively for private repositories.

Quick Reference — One-Line Install (Ubuntu 22.04 / 24.04)

```
# Step 1: system packages
sudo apt-get update && sudo apt-get install -y \
    bash git curl jq zip unzip file coreutils wget gpg \
    python3 python3-pip python3-venv build-essential \
    nodejs npm shellcheck cppcheck clamav clamav-daemon yara

# Step 2: Python venv + packages
python3 -m venv /opt/ai-transit/venv
source /opt/ai-transit/venv/bin/activate
pip install --upgrade pip
pip install openpyxl reportlab detect-secrets bandit pip-audit safety \
    semgrep checkov scancode-toolkit

# Step 3: gitleaks (needs Go 1.24.11 or newer from https://go.dev/dl/, not the distribution package)
CGO_ENABLED=0 go install github.com/zricethezav/gitleaks/v8@latest
sudo cp ~/go/bin/gitleaks /usr/local/bin/gitleaks

# Step 4: trivy
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
    | sudo sh -s -- -b /usr/local/bin

# Step 5: hadolint
VERSION=$(curl -s https://api.github.com/repos/hadolint/hadolint/releases/latest \
    | jq -r '.tag_name')
```

```
sudo curl -sSL \
    "https://github.com/hadolint/hadolint/releases/download/${VERSION}/hadolint-Linux-x86_64" \
    -o /usr/local/bin/hadolint && sudo chmod +x /usr/local/bin/hadolint

# Step 6: ClamAV DB update
sudo freshclam

# Step 7: trivy DB
trivy image --download-db-only

echo "Installation complete — run ./verify_install.sh to confirm"
```

AI Transit Pipeline — Installation Guide v2.1 — 2025